

Lecture 8: Logical Agents Part I

Russell and Norvig: Chapter 7

CS-4820/5820

Tu/Th 12:15 PM-1:30 PM

Room: Centennial Hall 106

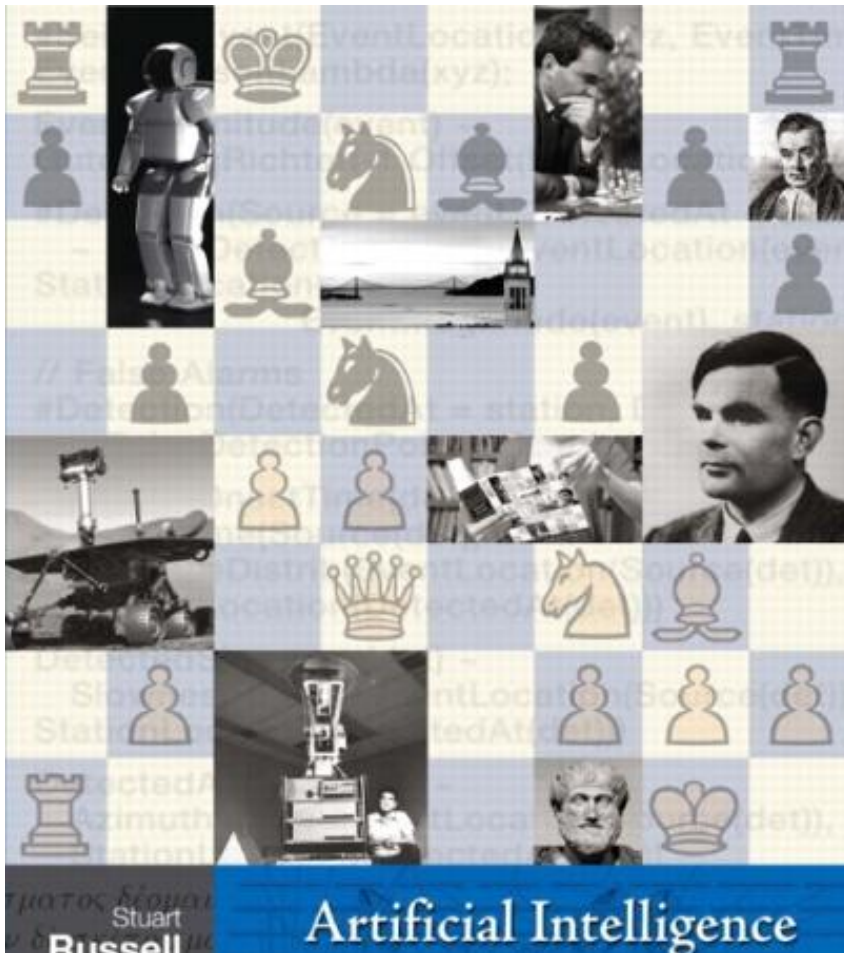
Instructor: Adham Atyabi

Office: Engineering 243

Office Hours: Mon 9:00 AM-14:00 PM.

Email: aatyabi@uccs.edu

**Teaching Assistant: Ali Al Shami
(aalshami@uccs.edu)**



Outline

- Knowledge-based agents
- Wumpus world
- Logic in general - models and entailment
- Propositional (Boolean) logic
- Equivalence, validity, satisfiability
- Inference rules and theorem proving
 - forward chaining
 - backward chaining
 - resolution

Logical Agent

- What is a logical agent?
- We can tell it facts or ask for inference
- For example
 - You can tell the agent the father of john is Bob
 - You can tell the agent the jane is john's sister
 - You can tell the agent the john's father is the same as john's sister's father
 - You can ask the agent who is jane's father

Knowledge based Agents

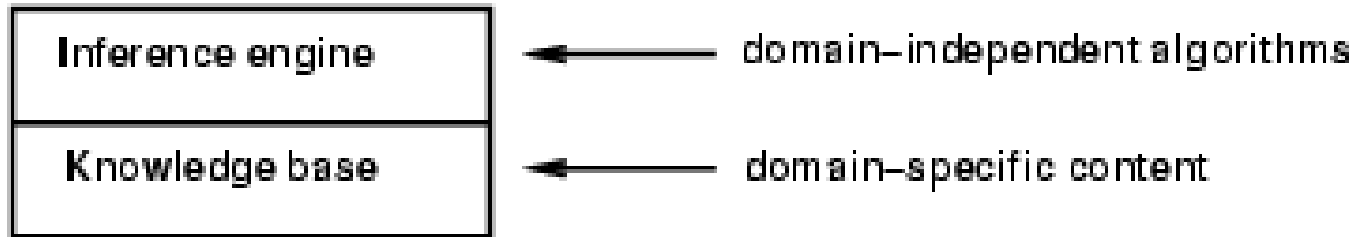
the main components of a knowledge-based system are:

- knowledge base
- Knowledge base representation language
- Inference engine
- have background knowledge about the world

Knowledge based Agents

- At every step:
 - Construct a sentence with assertion about percepts
 - Constructs a sentence asking what action is next
 - Constructs a sentence asserting that action

Knowledge bases



- Knowledge base = set of **sentences** in a **formal** language
- **Declarative** approach to building an agent (or other system):
 - Tell it what it needs to know
- Then it can Ask itself what to do - answers should follow from the Knowledge Base (KB)
- Agents can be viewed at the **knowledge level**
i.e., what they know, regardless of how implemented
- Or at the **implementation level**
 - i.e., data structures in KB and algorithms that manipulate them

A simple knowledge-based agent

```
function KB-AGENT(percept) returns an action  
  static: KB, a knowledge base  
           t, a counter, initially 0, indicating time  
  
  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))  
  action  $\leftarrow$  ASK(KB, MAKE-ACTION-QUERY(t))  
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))  
  t  $\leftarrow$  t + 1  
  return action
```

- The agent must be able to:
 - Represent states, actions, etc.
 - Incorporate new percepts
 - Update internal representations of the world
 - Deduce hidden properties of the world
 - Deduce appropriate actions

Wumpus World PEAS description

- **Performance measure**

- gold +1000, death -1000
- -1 per step, -10 for using the arrow

- **Environment**

- Squares adjacent to Wumpus are smelly
- Squares adjacent to pit are breezy
- Glitter if gold is in the same square
- Shooting kills Wumpus if you are facing it
- Shooting uses up the only arrow
- Grabbing picks up gold if in same square
- Releasing drops the gold in same square

- **Sensors:** Stench, Breeze, Glitter, Bump, Scream

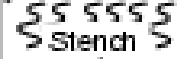
- **Actuators:** Left turn, Right turn, Forward, Grab, Release, Shoot

4

3

2

1

			
	  		
			
 START			

1

2

3

4

Wumpus world characterization

- Fully_Observable No – only **local** perception
- Deterministic Yes – outcomes exactly specified
- Episodic No – sequential at the level of actions
- Static Yes – Wumpus and Pits do not move
- Discrete Yes
- Single-agent? Yes – Wumpus is essentially a natural feature

Exploring a wumpus world

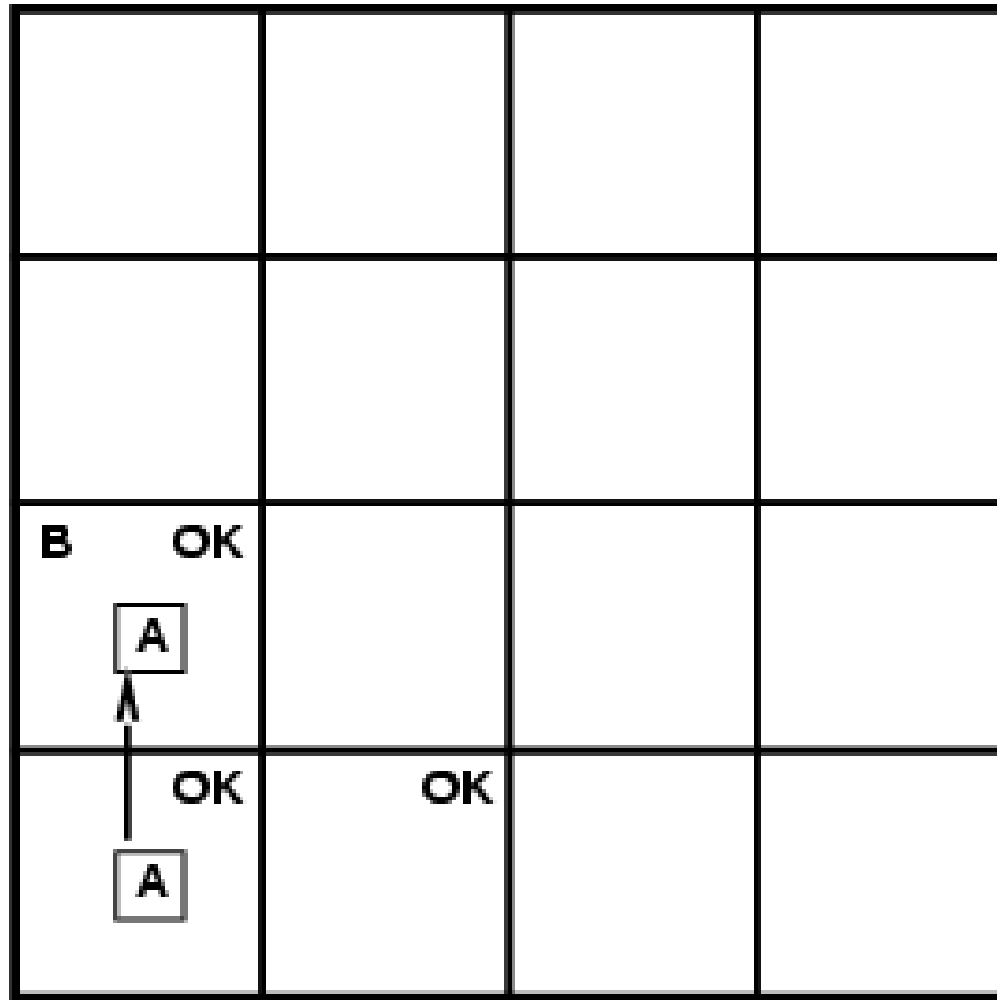
1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

A

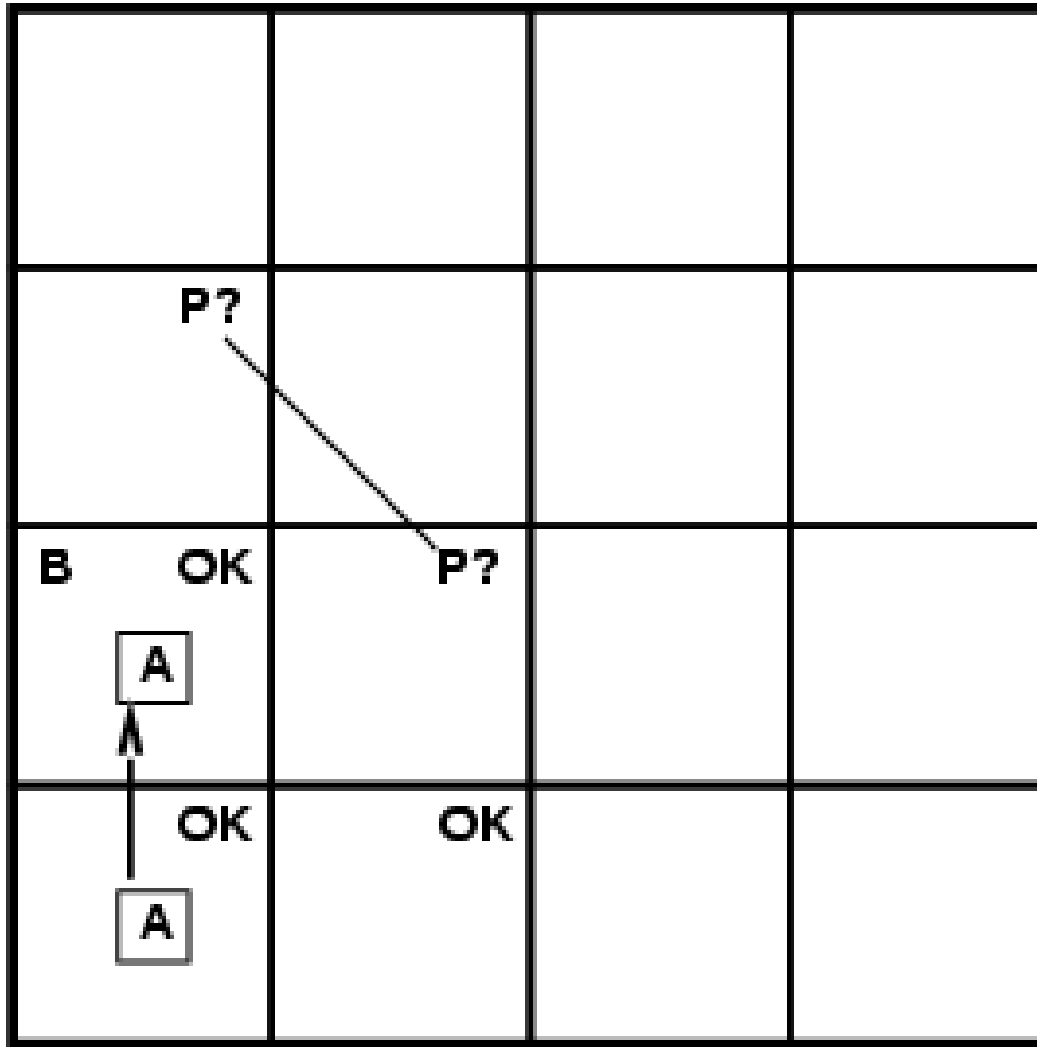
 = Agent
 B = Breeze
 G = Glitter, Gold
 OK = Safe square
 P = Pit
 S = Stench
 V = Visited
 W = Wumpus

[None,None,None,None,None]

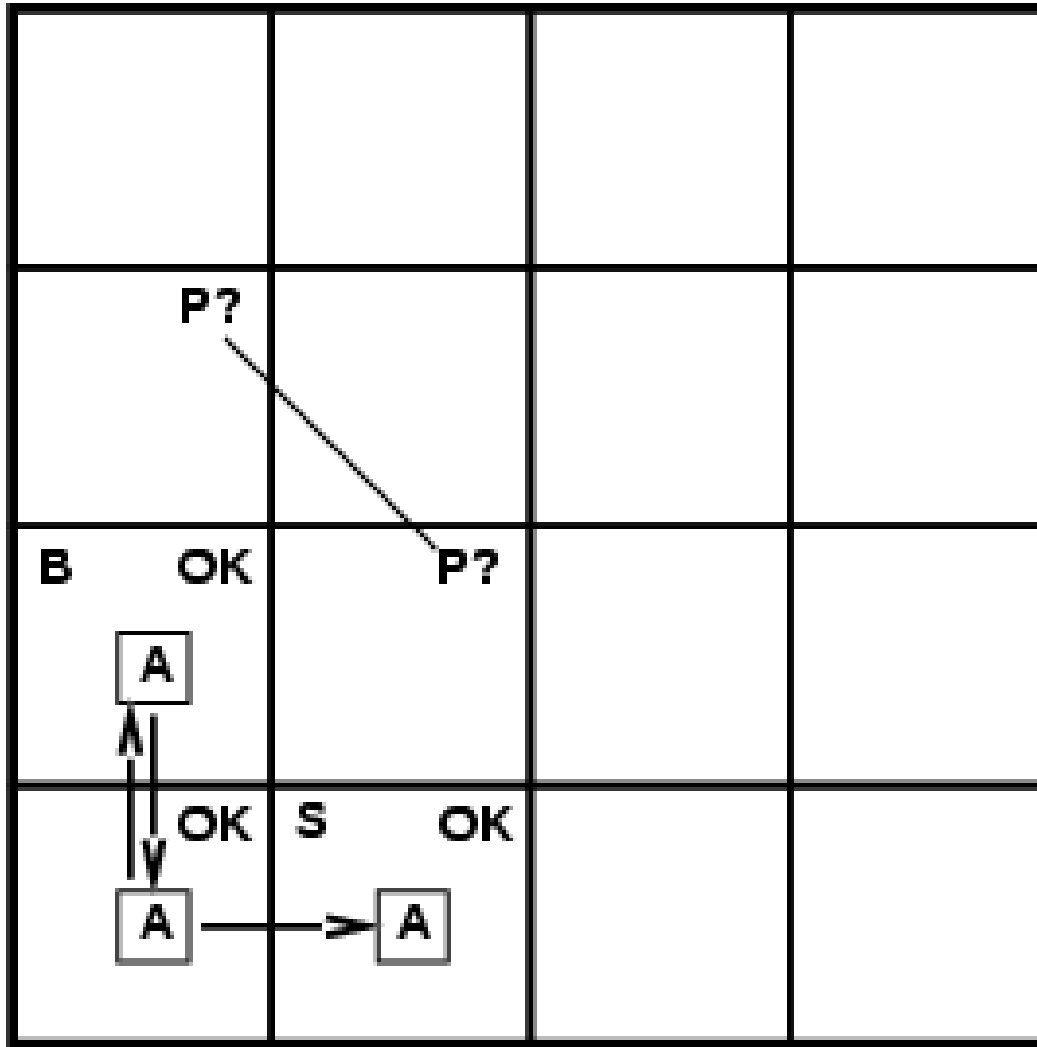
Exploring a wumpus world



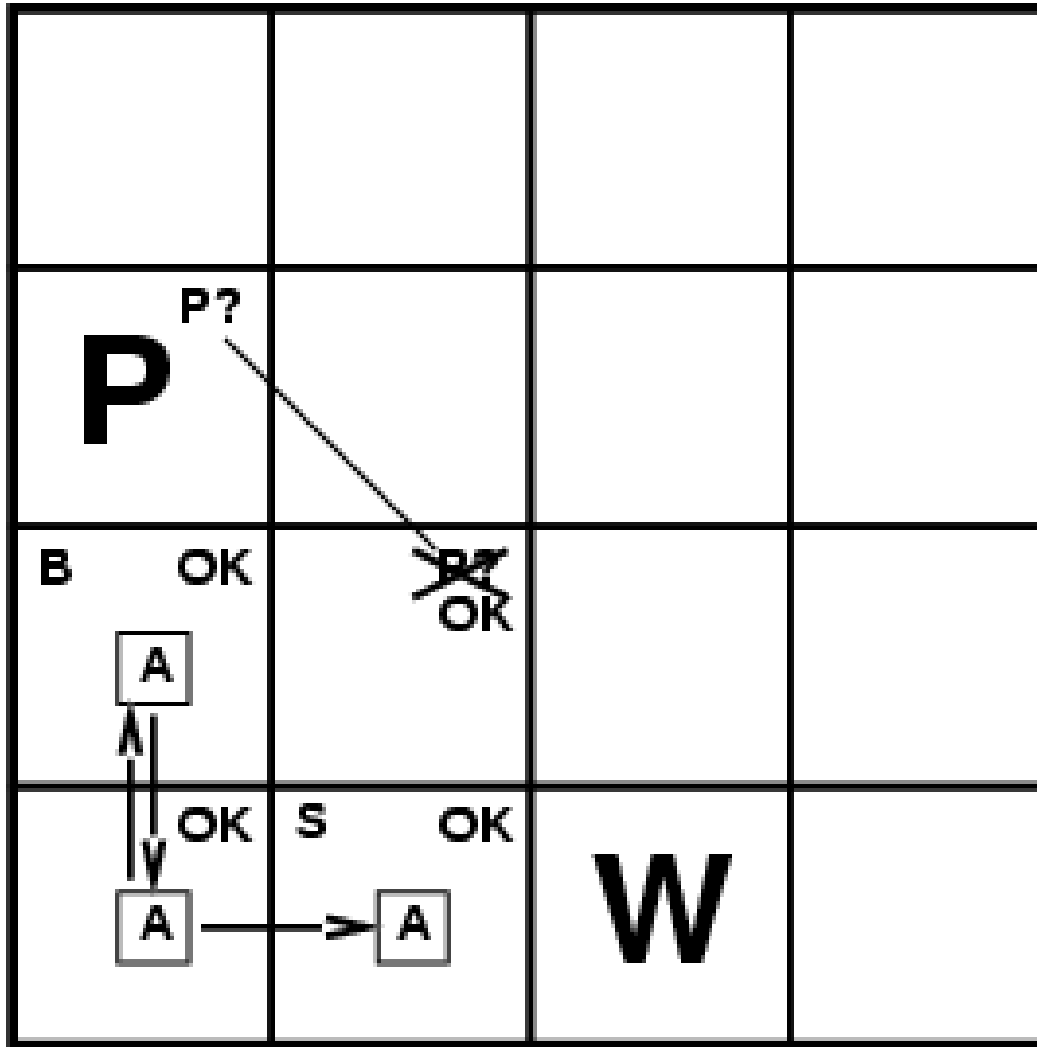
Exploring a wumpus world



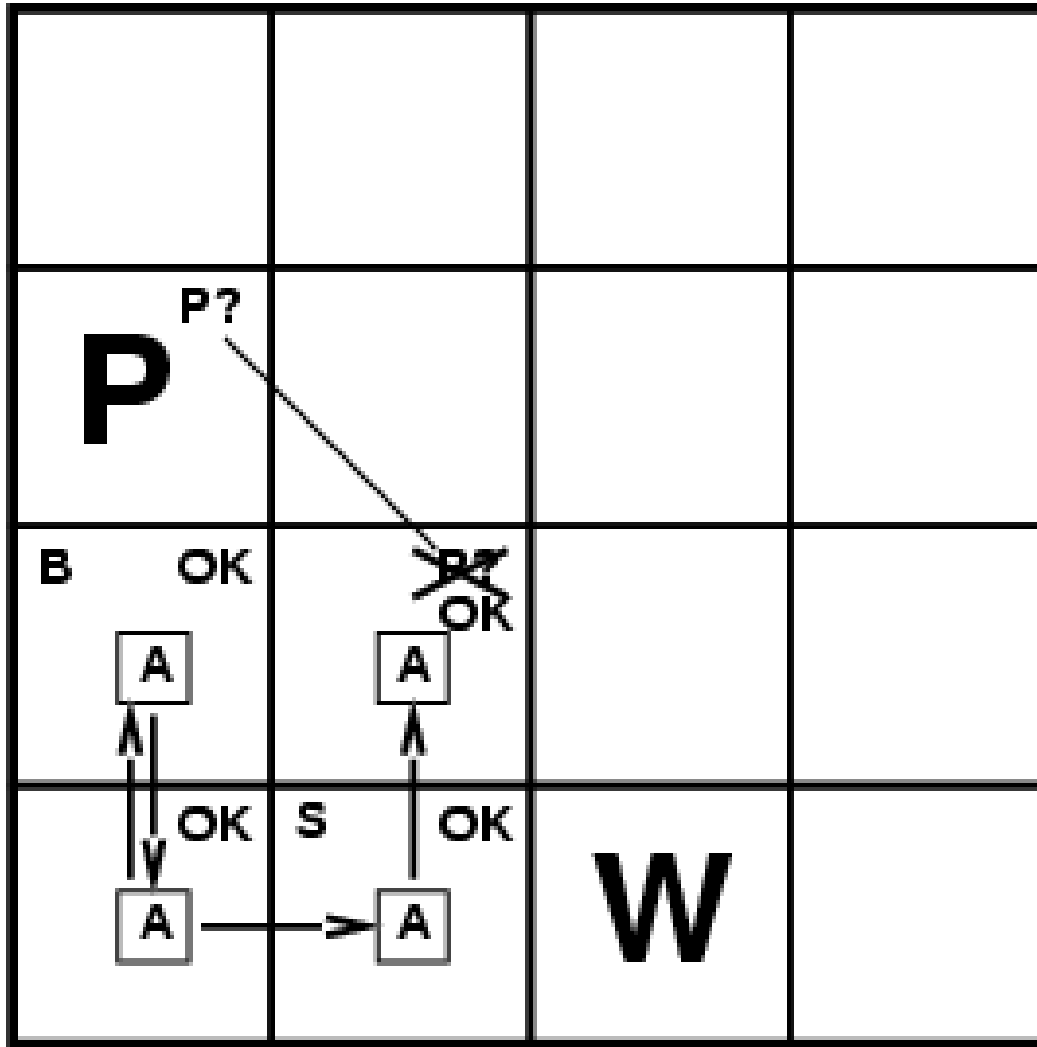
Exploring a wumpus world



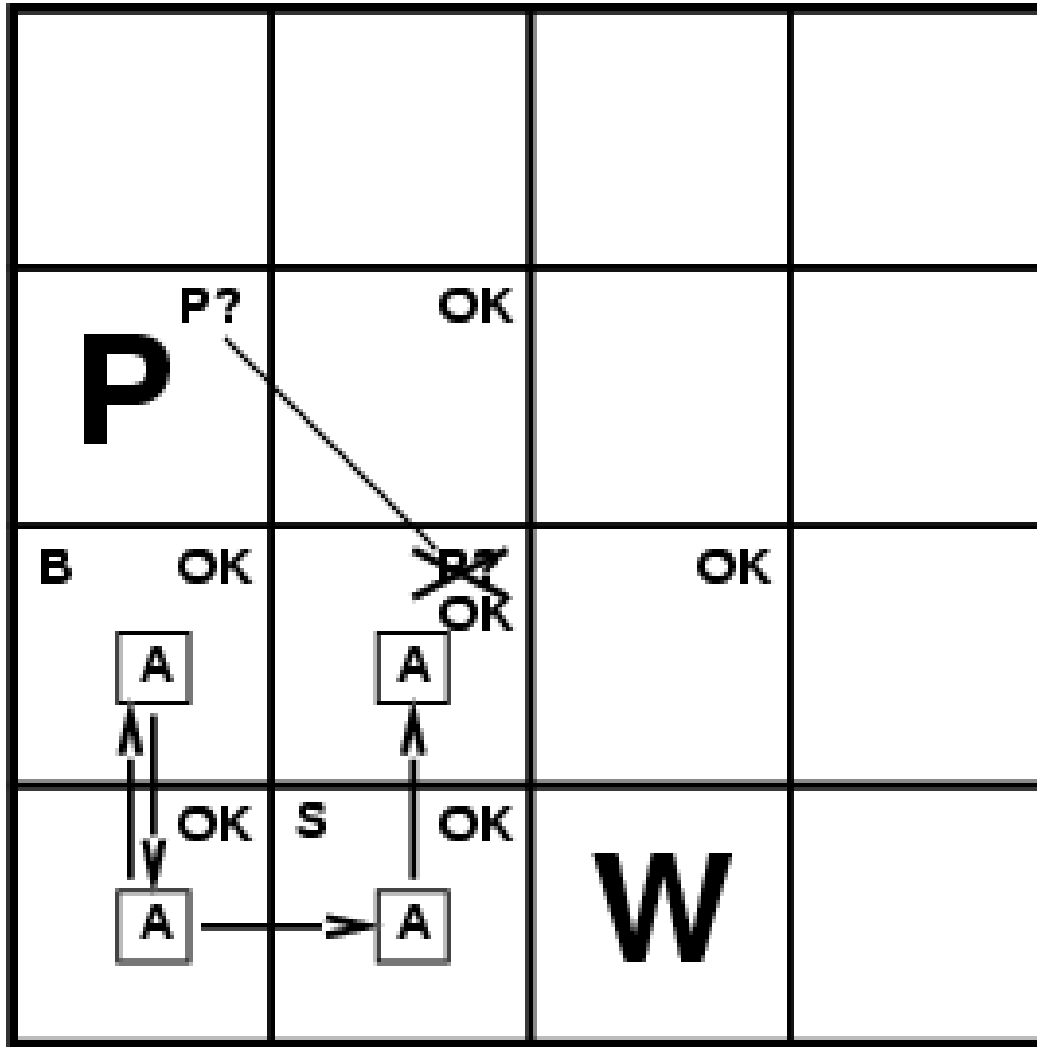
Exploring a wumpus world



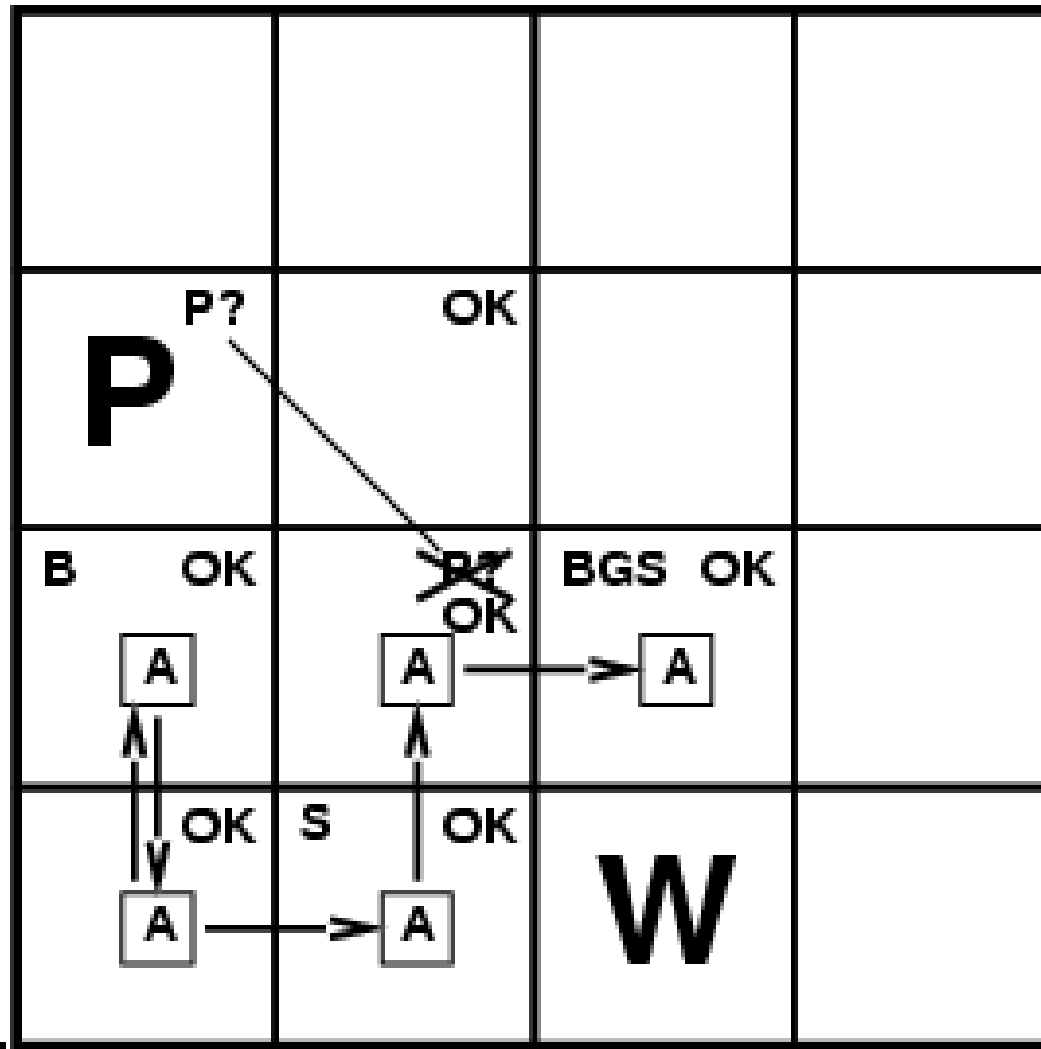
Exploring a wumpus world



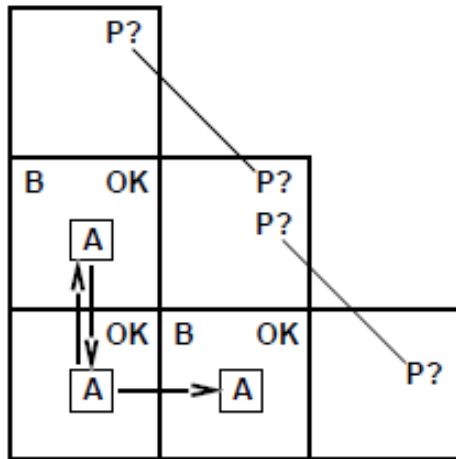
Exploring a wumpus world



Exploring a wumpus world

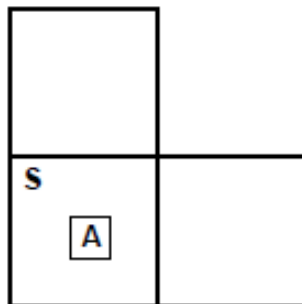


Other tight spots



Breeze in (1,2) and (2,1)
 $\Rightarrow$ no safe actions

Assuming pits uniformly distributed,
 (2,2) has pit w/ prob 0.86, vs. 0.31



Smell in (1,1)

$\Rightarrow$ cannot move

Can use a strategy of coercion:

shoot straight ahead

wumpus was there $\Rightarrow$ dead $\Rightarrow$ safe

wumpus wasn't there $\Rightarrow$ safe

Logic in general

- **Logics** are formal languages for representing information such that conclusions can be drawn
- **Syntax** defines the sentences in the language
 - $x+4=6$
 - $4x=6+$
- **Semantics** define the "meaning" of sentences;
 - i.e., define **truth** of a sentence in a world
- The **models** describe the possible world.
- $M(\alpha)$ is the set of all models of α . If sentence α is **true** for model m , α satisfies m .
- E.g., the language of arithmetic
 - $x+2 \geq y$ is a sentence; $x+2+y > \{ \}$ is not a sentence
 - $x+2 \geq y$ is true if the number $x+2$ is no less than the number y
 - $x+2 \geq y$ is true in a world where $x = 7, y = 1$
 - $x+2 \geq y$ is false in a world where $x = 0, y = 6$

Entailment

- **Entailment** means that one thing **follows from** another:

$$KB \models \alpha$$

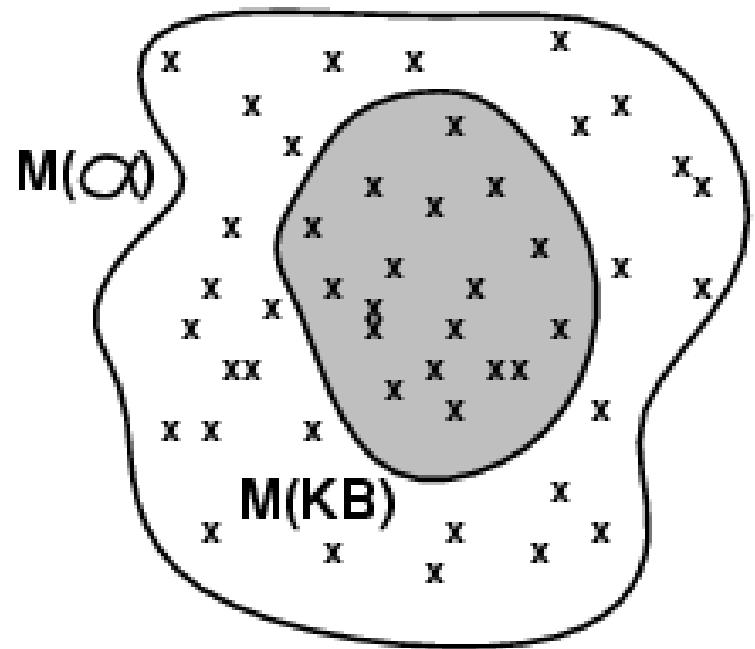
$$KB \models \alpha \text{ iff } M(\alpha) \subseteq M(KB).$$

Knowledge base *KB* entails sentence α if and only if α is true in all worlds where *KB* is true

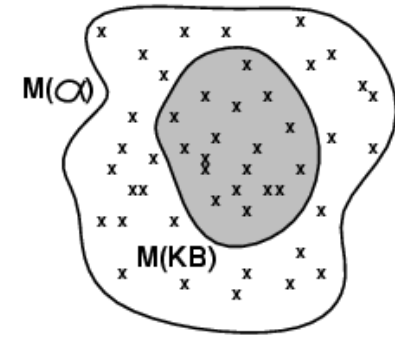
- E.g., the KB containing “the Giants won” and “the Reds won” entails “Either the Giants won or the Reds won”
- E.g., $x+y = 4$ entails $4 = x+y$
- Entailment is a relationship between sentences (i.e., **syntax**) that is based on **semantics**

Models

- Logicians typically think in terms of **models**, which are formally structured worlds with respect to which truth can be evaluated
- We say m **is a model of** a sentence α if α is true in m
- $M(\alpha)$ is the set of all models of α
- Then $KB \models \alpha$ if $M(KB) \subseteq M(\alpha)$
 - $KB = \text{Giants won and Reds won}$
 $\alpha = \text{Giants won}$
 - $(x=0) \models (xy=0)$
 - $(p = \text{true}) \models (p \vee q) = \text{true}$

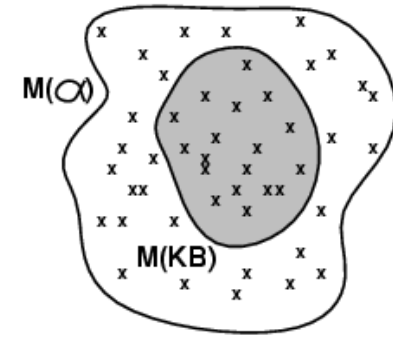


Models



- $KB \models \alpha$: represents a fundamental concept in the realm of logic and knowledge bases, signifying that a knowledge base (KB) entails a statement (α).
- KB is a collection of known facts, assertions, and rules about a domain of interest.
- $\models$ represents a statement, proposition, or assertion that we are checking to see if it can be logically inferred from the knowledge base.
- $KB \models \alpha$ reads as "KB entails α ," or " α is entailed by KB," meaning that whenever the facts and rules in KB are true, then α must also be true. This is a way to say that α is a logical consequence of the knowledge base KB.

Models



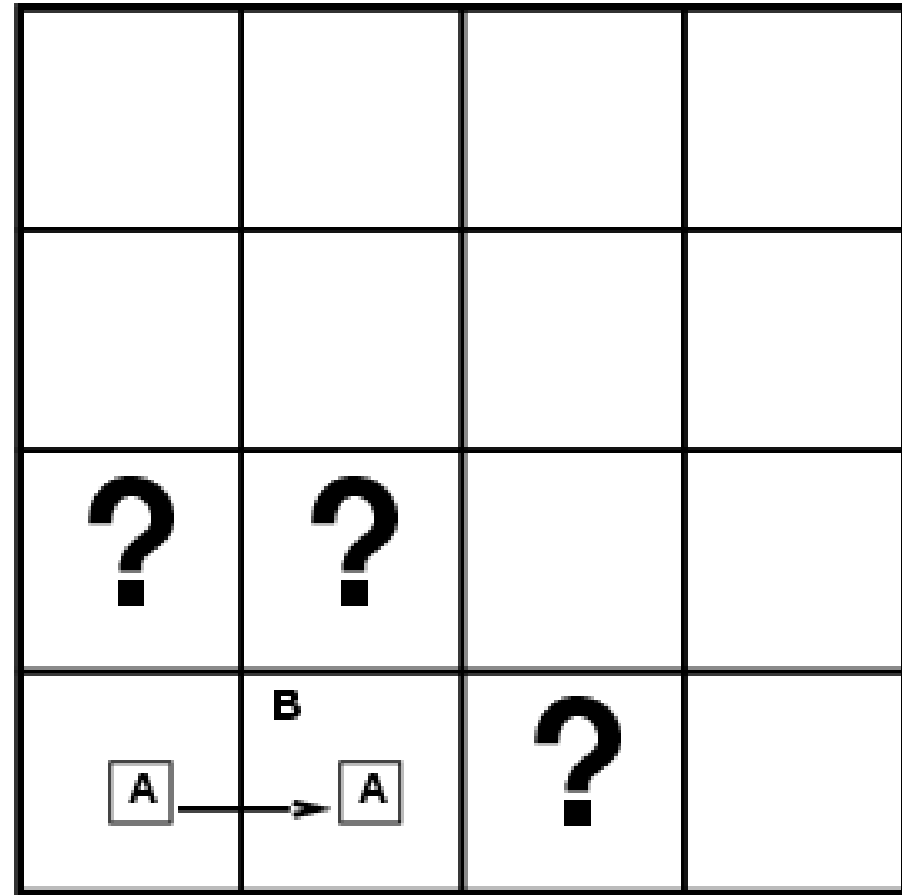
- $KB \models \alpha$ if $M(KB) \subseteq M(\alpha)$
- $M(KB) \subseteq M(\alpha)$
- $M(KB)$: Represents the set of all models (interpretations of the world) where the knowledge base KB holds true. A model is a specific assignment of truth values to statements that makes all statements in KB true.
- $M(\alpha)$: Represents the set of all models where the statement α holds true.
- The expression " $M(KB) \subseteq M(\alpha)$ " means that every model in which the knowledge base KB is true is also a model in which the statement α is true. In simpler terms, **there's no possible interpretation of the world that makes all statements in KB true without also making α true**. This is the model-theoretic way of expressing that the knowledge base KB entails the statement α .
- " $KB \models \alpha$ iff $M(KB) \subseteq M(\alpha)$ " expresses the concept of entailment both logically and model-theoretically.
- α is a logical consequence of a knowledge base KB if and only if every model (interpretation) that makes KB true also makes α true.

Entailment in the wumpus world

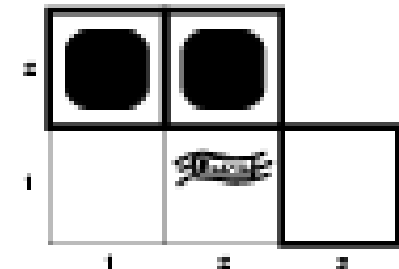
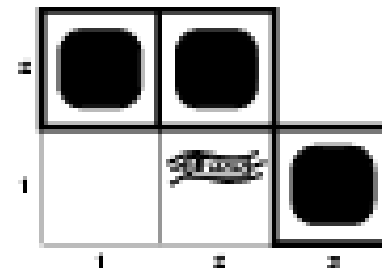
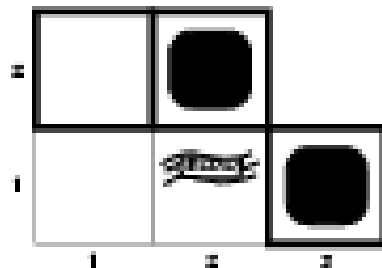
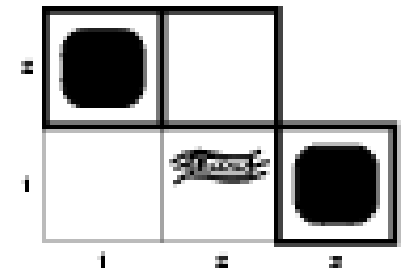
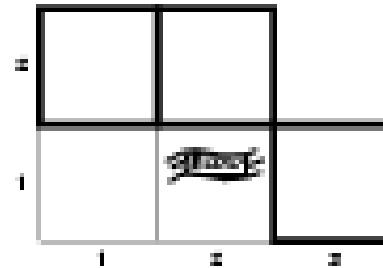
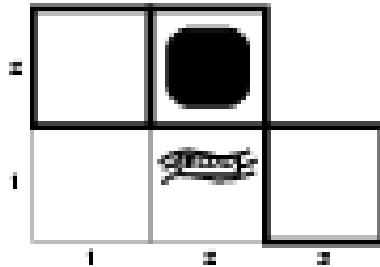
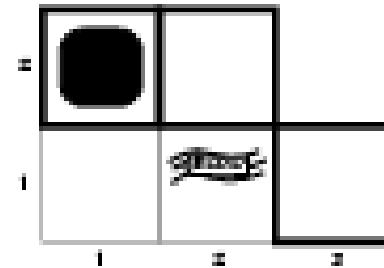
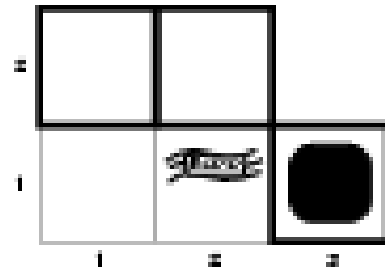
Situation after detecting
nothing in [1,1], moving
right, breeze in [2,1]

Consider possible models for
KB assuming only pits

3 Boolean choices $\Rightarrow$ 8
possible models



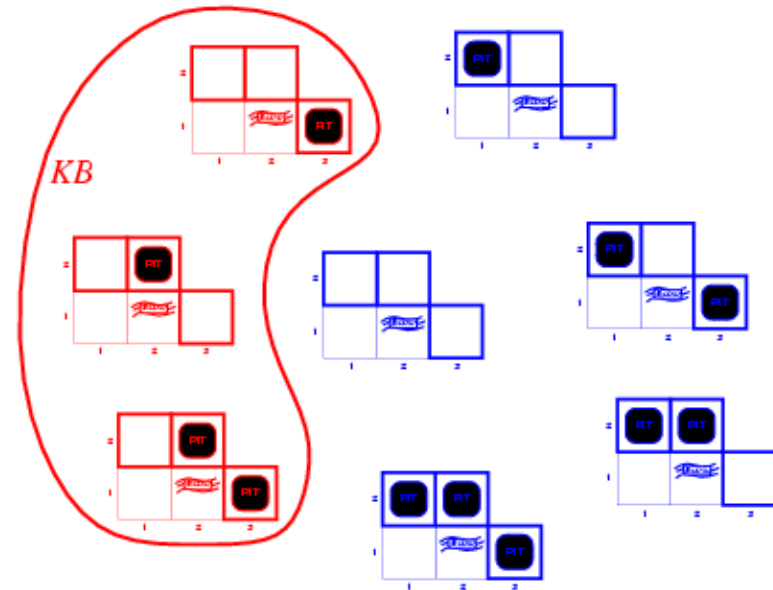
Wumpus models



Wumpus models

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
OK			
1,1	2,1	3,1	4,1
A			
OK	OK		

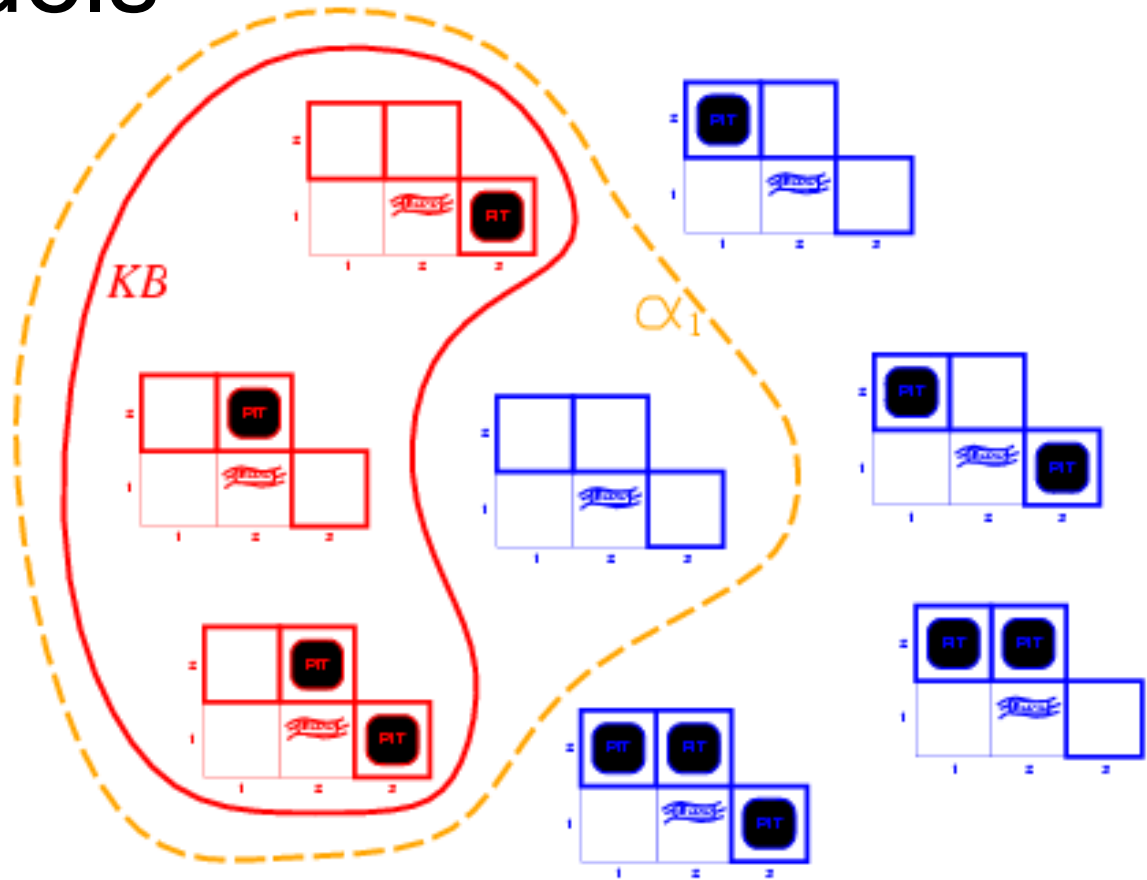
A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus



[None, None, None, None, None]

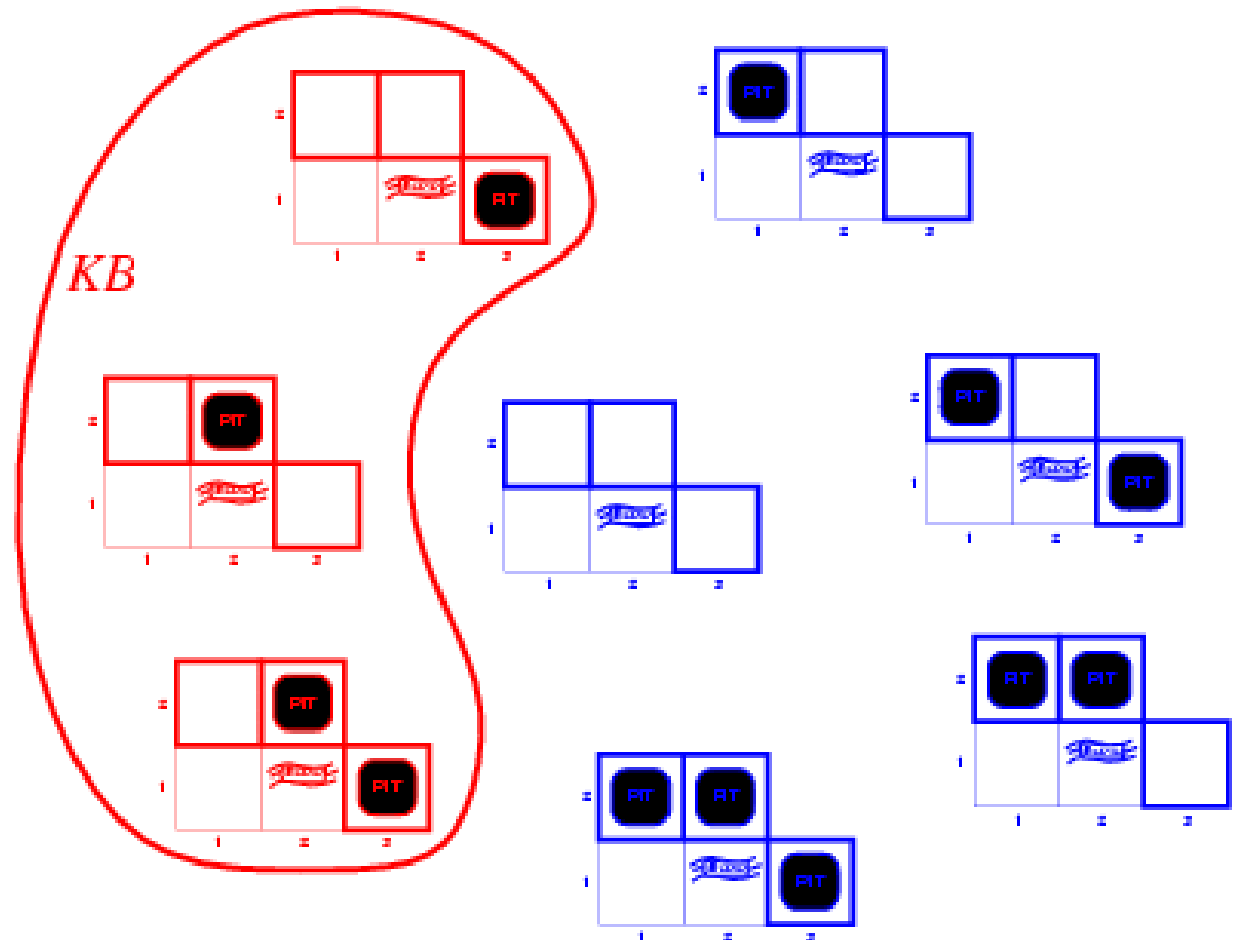
$KB = \text{Wumpus-world rules} + \text{observations}$

Wumpus models



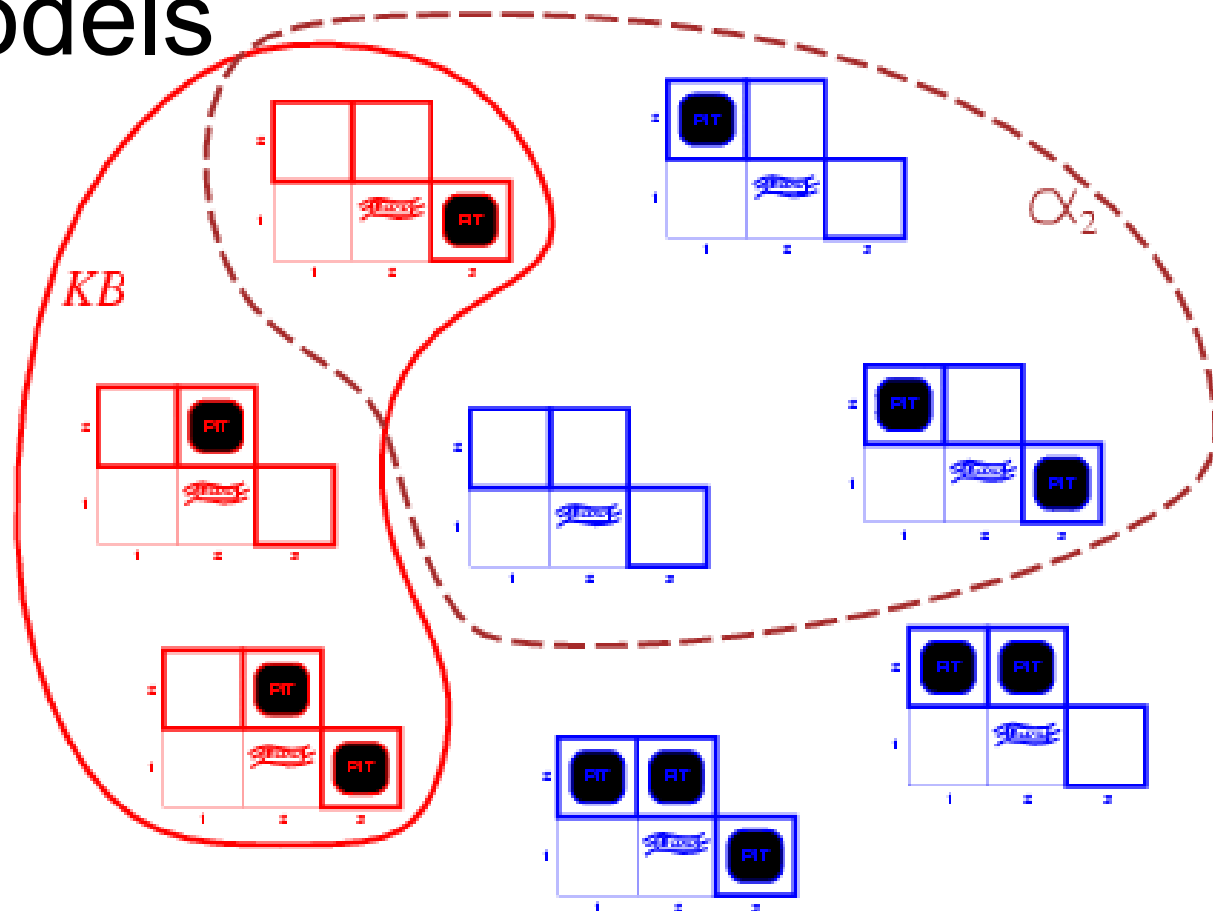
- KB = Wumpus-world rules + observations
- α_1 = "[1,2] is safe", $KB \models \alpha_1$, proved by **model checking**

Wumpus models



- KB = Wumpus-world rules + observations

Wumpus models



- KB = Wumpus-world rules + observations
- α_2 = "[2,2] is safe", $KB \models \alpha_2$

Inference

- $KB \vdash_i \alpha$ = sentence α can be derived from KB by procedure i
- **Soundness**: i is sound if whenever $KB \vdash_i \alpha$, it is also true that $KB \models \alpha$
- **Completeness**: i is complete if whenever $KB \models \alpha$, it is also true that $KB \vdash_i \alpha$
- **Preview**: we will define a logic (first-order logic) which is expressive enough to say almost anything of interest, and for which there exists a sound and complete inference procedure.
- That is, the procedure will answer any question whose answer follows from what is known by the KB .

Propositional logic: Syntax

- Propositional logic is the simplest logic – illustrates basic ideas
- The proposition symbols P_1, P_2 etc are sentences
 - If S is a sentence, $\neg S$ is a sentence (**negation**)
 - If S_1 and S_2 are sentences, $S_1 \wedge S_2$ is a sentence (**conjunction**)
 - If S_1 and S_2 are sentences, $S_1 \vee S_2$ is a sentence (**disjunction**)
 - If S_1 and S_2 are sentences, $S_1 \Rightarrow S_2$ is a sentence (**implication**)
 - If S_1 and S_2 are sentences, $S_1 \Leftrightarrow S_2$ is a sentence (**biconditional**)

Propositional logic: Semantics

Each model specifies true/false for each proposition symbol

E.g. $P_{1,2}$ false $P_{2,2}$ true $P_{3,1}$ false

With these symbols, 8 possible models, can be enumerated automatically.

Rules for evaluating truth with respect to a model m :

$\neg S$	is true if	S is false	
$S_1 \wedge S_2$	is true if	S_1 is true and	S_2 is true
$S_1 \vee S_2$	is true if	S_1 is true or	S_2 is true
$S_1 \Rightarrow S_2$	is true if	S_1 is false or	S_2 is true
i.e.,	is false if	S_1 is true and	S_2 is false
$S_1 \Leftrightarrow S_2$	is true if	$S_1 \Rightarrow S_2$ is true and	$S_2 \Rightarrow S_1$ is true

Simple recursive process evaluates an arbitrary sentence, e.g.,

$$\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1}) = \text{true} \wedge (\text{true} \vee \text{false}) = \text{true} \wedge \text{true} = \text{true}$$

Truth tables for connectives

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

Wumpus world sentences

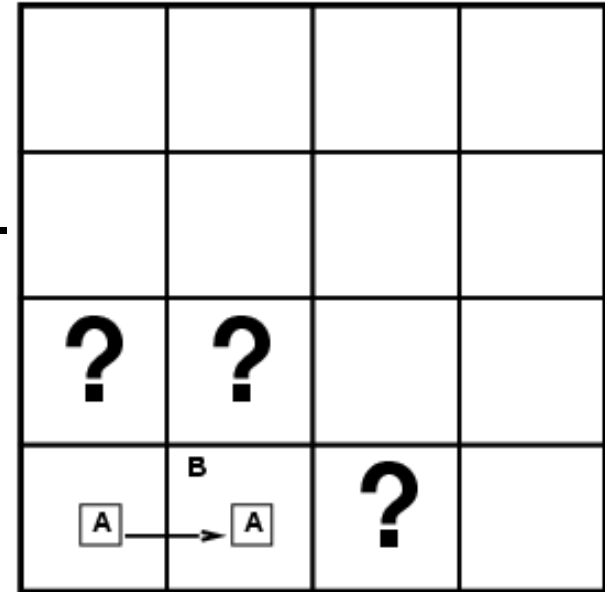
Let $P_{i,j}$ be true if there is a pit in $[i, j]$.

Let $B_{i,j}$ be true if there is a breeze in $[i, j]$.

$$\neg P_{1,1}$$

$$\neg B_{1,1}$$

$$B_{2,1}$$



- "Pits cause breezes in adjacent squares"

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

$$B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$

Truth tables for inference

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	KB	α_1
false	false	false	false	false	false	false	false	true
false	false	false	false	false	false	true	false	true
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
false	true	false	false	false	false	false	false	true
false	true	false	false	false	false	true	<u>true</u>	<u>true</u>
false	true	false	false	false	true	false	<u>true</u>	<u>true</u>
false	true	false	false	false	true	true	<u>true</u>	<u>true</u>
false	true	false	false	true	false	false	false	true
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
true	true	true	true	true	true	true	false	false

Inference by enumeration

- Depth-first enumeration of all models is sound and complete

```
function TT-ENTAILS?(KB,  $\alpha$ ) returns true or false
```

```
  symbols  $\leftarrow$  a list of the proposition symbols in KB and  $\alpha$ 
```

```
  return TT-CHECK-ALL(KB,  $\alpha$ , symbols, [])
```

```
function TT-CHECK-ALL(KB,  $\alpha$ , symbols, model) returns true or false
```

```
  if EMPTY?(symbols) then
```

```
    if PL-TRUE?(KB, model) then return PL-TRUE?( $\alpha$ , model)
```

```
    else return true
```

```
  else do
```

```
    P  $\leftarrow$  FIRST(symbols); rest  $\leftarrow$  REST(symbols)
```

```
    return TT-CHECK-ALL(KB,  $\alpha$ , rest, EXTEND(P, true, model)) and
```

```
      TT-CHECK-ALL(KB,  $\alpha$ , rest, EXTEND(P, false, model))
```

- For n symbols, time complexity is $O(2^n)$, space complexity is $O(n)$

Logical equivalence

- Two sentences are **logically equivalent** iff true in same models: $\alpha \equiv \beta$ iff $\alpha \models \beta$ and $\beta \models \alpha$

$$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha) \quad \text{commutativity of } \wedge$$

$$(\alpha \vee \beta) \equiv (\beta \vee \alpha) \quad \text{commutativity of } \vee$$

$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma)) \quad \text{associativity of } \wedge$$

$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma)) \quad \text{associativity of } \vee$$

$$\neg(\neg\alpha) \equiv \alpha \quad \text{double-negation elimination}$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha) \quad \text{contraposition}$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta) \quad \text{implication elimination}$$

$$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) \quad \text{biconditional elimination}$$

$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \quad \text{de Morgan}$$

$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \quad \text{de Morgan}$$

$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) \quad \text{distributivity of } \wedge \text{ over } \vee$$

$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) \quad \text{distributivity of } \vee \text{ over } \wedge$$

Validity and satisfiability

Logical Equivalence: $\alpha \equiv \beta$ iff $\alpha \models \beta$ and $\beta \models \alpha$

A sentence is **valid** if it is true in **all** models,
e.g., *True*, $A \vee \neg A$, $A \Rightarrow A$, $(A \wedge (A \Rightarrow B)) \Rightarrow B$

Validity is connected to inference via the **Deduction Theorem**:
 $KB \models \alpha$ if and only if $(KB \Rightarrow \alpha)$ is valid

A sentence is **satisfiable** if it is true in **some** model
e.g., $A \vee B$,

A sentence is **unsatisfiable** if it is true in **no** models
e.g., $A \wedge \neg A$

Satisfiability is connected to inference via the following:
 $KB \models \alpha$ if and only if $(KB \wedge \neg \alpha)$ is unsatisfiable

Proof methods

- Proof methods divide into (roughly) two kinds:
 - Application of inference rules
 - *Legitimate (sound) generation of new sentences from old*
 - **Proof** = a sequence of inference rule applications
Can use inference rules as operators in a standard search algorithm
 - *Typically require transformation of sentences into a normal form*
 - Model checking
 - *truth table enumeration (always exponential in n)*
 - *improved backtracking, e.g., Davis--Putnam-Logemann-Loveland (DPLL)*
 - *heuristic search in model space (sound but incomplete)*
e.g., min-conflicts-like hill-climbing algorithms

Inference in Propositional Logic

$$R_1 : \neg P_{1,1}$$

$$R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

$$R_3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$

$$R_4 : \neg B_{1,1}$$

$$R_5 : B_{2,1}$$

$$R_6 : (B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

$$R_7 : ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}) \quad \text{(a): moving back to [1,2] from [2,1] stench and no breeze:}$$

$$R_8 : (\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1}))$$

$$R_9 : \neg(P_{1,2} \vee P_{2,1})$$

$$R_{10} : \neg P_{1,2} \wedge \neg P_{2,1}$$

similar prior inference:
biconditional elimination on R_3
and Modus Ponens on R_5 :
resolution on R_{13} and R_{15} :
resolution on R_1 and R_{16} :

1,4	2,4	3,4	4,4
1,3 W?	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P?	4,1

(a)

A: Agent
B: Breeze
G: Gold
OK: safe
P: Pit
S: Stench
V: Visited
W: Wumpus

1,4	2,4 P?	3,4	4,4
1,3 W?	2,3 A S G B	3,3 P?	4,3
1,2 S V OK	2,2 V OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P?	4,1

(b)

$$R_{11} : \neg B_{1,2}$$

$$R_{12} : B_{1,2} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{1,3})$$

$$R_{13} : \neg P_{2,2}$$

$$R_{14} : \neg P_{1,3}$$

$$R_{15} : P_{1,1} \vee P_{2,2} \vee P_{3,1}$$

$$R_{16} : P_{1,1} \vee P_{3,1}$$

$$R_{17} : P_{3,1}$$

Resolution

Conjunctive Normal Form (CNF): conjunction of disjunctions of literals
clauses

E.g., $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

1. Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

2. Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg \alpha \vee \beta$.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

3. Move $\neg$ inwards using de Morgan's rules and double-negation:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$

4. Apply distributivity law ($\wedge$ over $\vee$) and flatten:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

Resolution

Conjunctive Normal Form (CNF): conjunction of disjunctions of literals
clauses

E.g., $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

- Unit resolution** inference rule (for CNF):

$$\frac{\ell_i \vee \dots \vee \ell_k, m_1}{\ell_i \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k}$$

where ℓ_i and m_j are complementary literals.

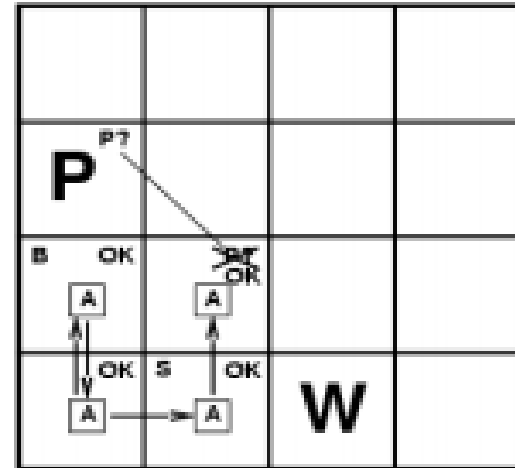
- Full Resolution** inference rule (for CNF):

$$\frac{\ell_i \vee \dots \vee \ell_k, m_1 \vee \dots \vee m_n}{\ell_i \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

where ℓ_i and m_j are complementary literals.

E.g., $\frac{P_{1,3} \vee P_{2,2}, \neg P_{2,2}}{P_{1,3}}$

- Resolution is sound and complete for propositional logic.



Resolution – Factoring

- Factoring is an inference rule that deals with eliminating redundancy within a single clause.
- The purpose of factoring is to simplify clauses that contain multiple instances of the same literal, possibly with different variables, by unifying those literals.

$$P(a,b) \vee P(x,y)$$

- If a and x , and b and y , can be unified (made identical by a substitution of variables), then the clause can be simplified.

$$(P(x) \vee Q(y) \vee P(z))$$

- In this clause, $P(x)$ and $P(z)$ can be unified by choosing x and z to be the same term. If we unify z to x , the clause becomes:

$$(P(x) \vee Q(y))$$

Resolution

- Factoring: Resulting clause should only contain one copy of a literal:
Elimination of multiple literal copies is called factoring (not necessary for set notation of clauses).

- Soundness of resolution inference rule:

$$\neg(l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k) \Rightarrow l_i$$

$$\neg m_j \Rightarrow (m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n)$$

$$\neg(l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k) \Rightarrow (m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n)$$

- Resolution is basis of a whole family of complete inference procedures:
 - Every complete search algorithm which uses resolution can derive any conclusion entailed by any knowledge base in PL.
- Resolution is **refutation complete**:
 - Given that A is true, It can be used to answer if $A \vee B$ is true but not to generate the consequence $A \vee B$

Propositional Resolution Propositional Resolution

- Resolution rule:

$$\frac{\alpha \vee \beta \quad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

Propositional Resolution Propositional Resolution

- Resolution rule:

$$\frac{\alpha \vee \beta \quad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

- Resolution refutation:

Propositional Resolution Propositional Resolution

- Resolution rule:

$$\frac{\alpha \vee \beta \quad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

- Resolution refutation:
 - Convert all sentences to CNF

Propositional Resolution Propositional Resolution

- Resolution rule:

$$\frac{\alpha \vee \beta \quad \neg \beta \vee \gamma}{\alpha \vee \gamma}$$

- Resolution refutation:
 - Convert all sentences to CNF
 - Negate the desired conclusion (converted to CNF)
 - Apply resolution rule until either
 - Derive false (a contradiction)
 - Can't apply any more

Propositional Resolution Propositional Resolution

- Resolution rule:

$$\frac{\begin{array}{l} \alpha \vee \beta \\ \neg\beta \vee \gamma \end{array}}{\alpha \vee \gamma}$$

- Resolution refutation:
 - Convert all sentences to CNF
 - Negate the desired conclusion (converted to CNF)
 - Apply resolution rule until either
 - Derive false (a contradiction)
 - Can't apply any more
- Resolution refutation is sound and complete
 - If we derive a contradiction, then the conclusion follows from the axioms
 - If we can't apply any more, then the conclusion cannot be proved from the axioms.

Resolution algorithm

- Proof by contradiction, i.e., show $KB \wedge \neg \alpha$ unsatisfiable

```
function PL-RESOLUTION( $KB, \alpha$ ) returns true or false
   $clauses \leftarrow$  the set of clauses in the CNF representation of  $KB \wedge \neg \alpha$ 
   $new \leftarrow \{ \}$ 
  loop do
    for each  $C_i, C_j$  in  $clauses$  do
       $resolvents \leftarrow$  PL-RESOLVE( $C_i, C_j$ )
      if  $resolvents$  contains the empty clause then return true
       $new \leftarrow new \cup resolvents$ 
    if  $new \subseteq clauses$  then return false
   $clauses \leftarrow clauses \cup new$ 
```

Resolution example

Say the agent is in [1,1], no breeze, so no pits can be in there

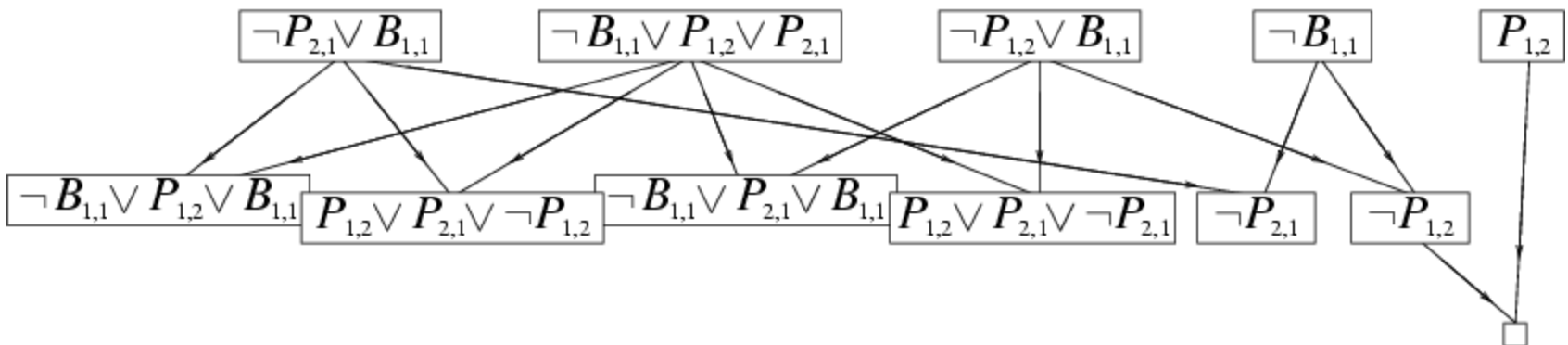
- $\alpha = \neg P_{1,2}$
- $KB = R_2 \wedge R_4$
- $KB = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1}$
- $KB \wedge \neg \alpha = \dots$

$$R_1 : \neg P_{1,1}$$

$$R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

$$R_3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$

$$R_4 : \neg B_{1,1}$$





University of Colorado
Colorado Springs



University of Colorado

Boulder | Colorado Springs | Denver | Anschutz Medical Campus

Lecture 8: Logical Agents Part II

Russell and Norvig: Chapter 7

CS-4820/5820

Tu/Th 12:15 PM-1:30 PM

Room: Centennial Hall 106

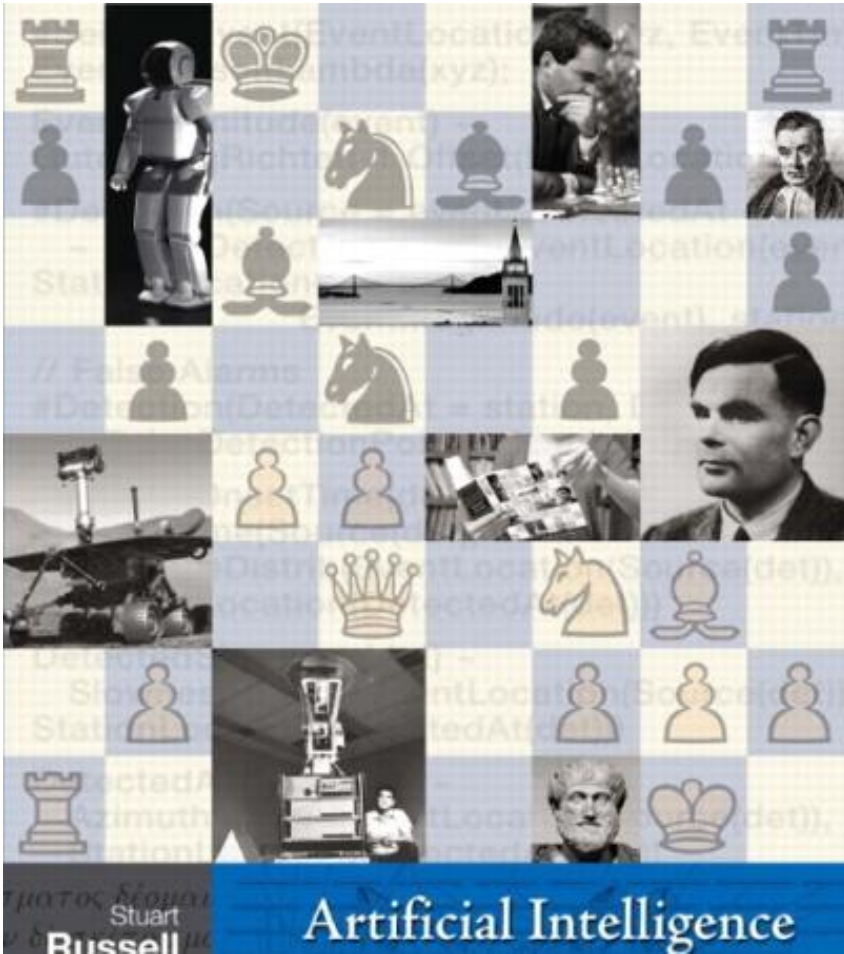
Instructor: Adham Atyabi

Office: Engineering 243

Office Hours: Mon 9:00 AM-14:00 PM.

Email: aatyabi@uccs.edu

Teaching Assistant: Ali Al Shami
(aalshami@uccs.edu)



Outline

- Knowledge-based agents
- Wumpus world
- Logic in general - models and entailment
- Propositional (Boolean) logic
- Equivalence, validity, satisfiability
- Inference rules and theorem proving
 - forward chaining
 - backward chaining
 - resolution

Resolution

Conjunctive Normal Form (CNF): conjunction of disjunctions of literals
clauses

E.g., $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})\beta$$

1. Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

2. Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

3. Move $\neg$ inwards using de Morgan's rules and double-negation:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \vee \neg P_{2,1}) \vee B_{1,1})$$

4. Apply distributivity law ($\wedge$ over $\vee$) and flatten:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

Resolution

Conjunctive Normal Form (CNF): conjunction of disjunctions of literals
clauses

E.g., $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

- Unit resolution** inference rule (for CNF):

$$\frac{\ell_i \vee \dots \vee \ell_k, m_1}{\ell_i \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k}$$

where ℓ_i and m_j are complementary literals.

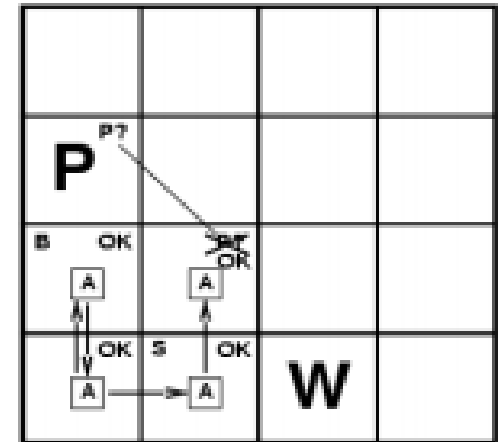
- Full Resolution** inference rule (for CNF):

$$\frac{\ell_i \vee \dots \vee \ell_k, m_1 \vee \dots \vee m_n}{\ell_i \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

where ℓ_i and m_j are complementary literals.

E.g., $\frac{P_{1,3} \vee P_{2,2}, \neg P_{2,2}}{P_{1,3}}$

- Resolution is sound and complete for propositional logic.



Forward and backward chaining

- **Horn Form** (restricted)

KB = **conjunction** of **Horn clauses**

- Horn clause (at most one literal is positive)
 - **proposition symbol (fact) or**
 - **(conjunction of symbols) $\Rightarrow$ symbol**
- E.g., $C \wedge (B \Rightarrow A) \wedge (C \wedge D \Rightarrow B)$
- $(\neg C \vee \neg B \vee A)$ becomes $(C \wedge B \Rightarrow A)$

- **Modus Ponens** (for Horn Form): complete for Horn KBs

$$\alpha_1, \dots, \alpha_n, \quad \alpha_1 \wedge \dots \wedge \alpha_n \Rightarrow \beta$$

$$\beta$$

- Can be used with **forward chaining** or **backward chaining**.
- These algorithms are very natural and run in **linear** time

Forward chaining algorithm

```
function PL-FC-ENTAILS?(KB, q) returns true or false
  local variables: count, a table, indexed by clause, initially the number of premises
                  inferred, a table, indexed by symbol, each entry initially false
                  agenda, a list of symbols, initially the symbols known to be true

  while agenda is not empty do
    p ← POP(agenda)
    unless inferred[p] do
      inferred[p] ← true
      for each Horn clause c in whose premise p appears do
        decrement count[c]
        if count[c] = 0 then do
          if HEAD[c] = q then return true
          PUSH(HEAD[c], agenda)

  return false
```

- Forward chaining is sound and complete for Horn KB

Forward chaining

- Idea: fire any rule whose premises are satisfied in the *KB*,
 - add its conclusion to the *KB*, until query is found

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

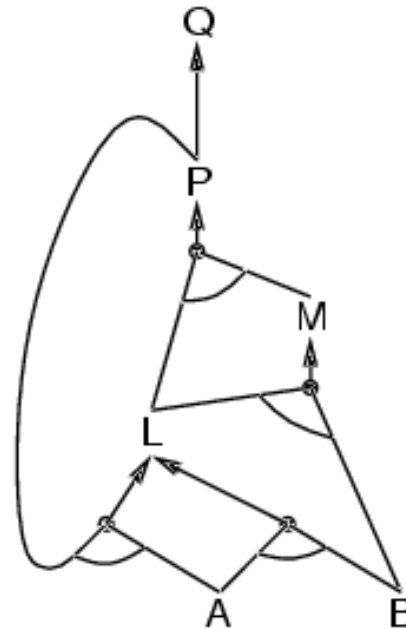
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Forward chaining example

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

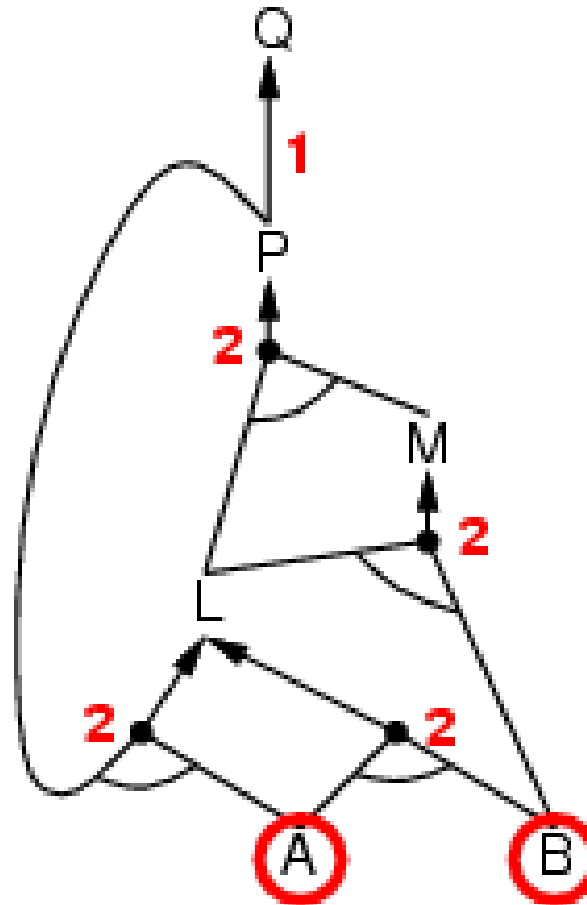
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Forward chaining example

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

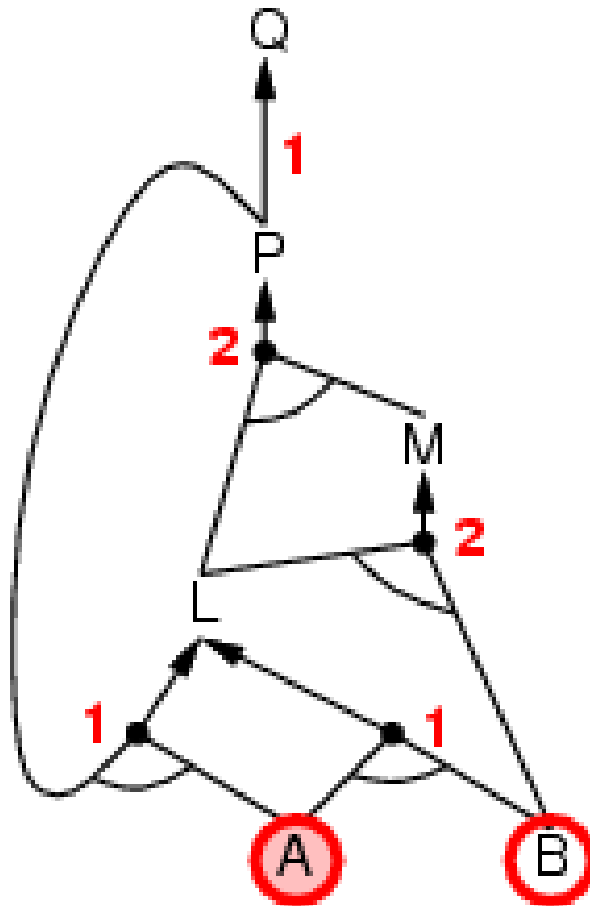
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Forward chaining example

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

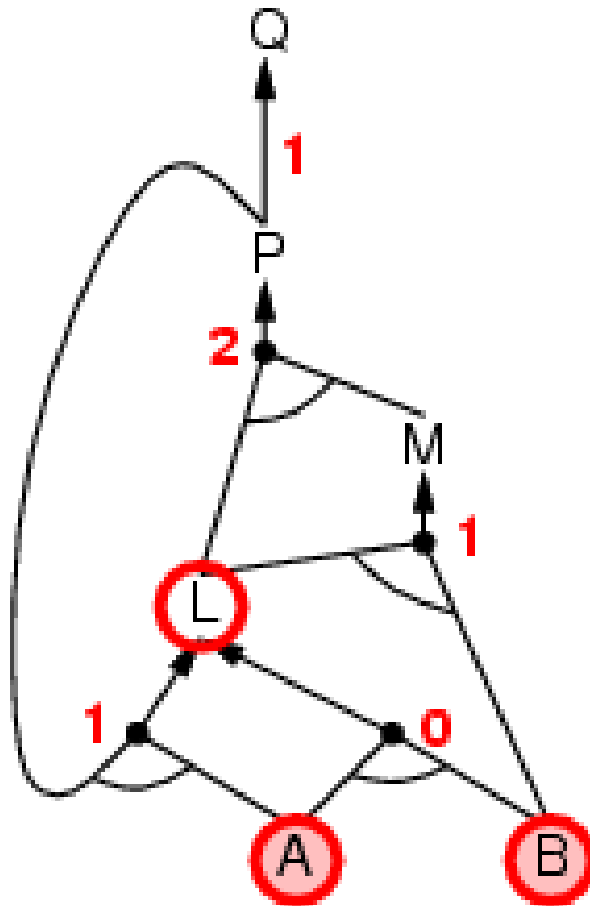
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

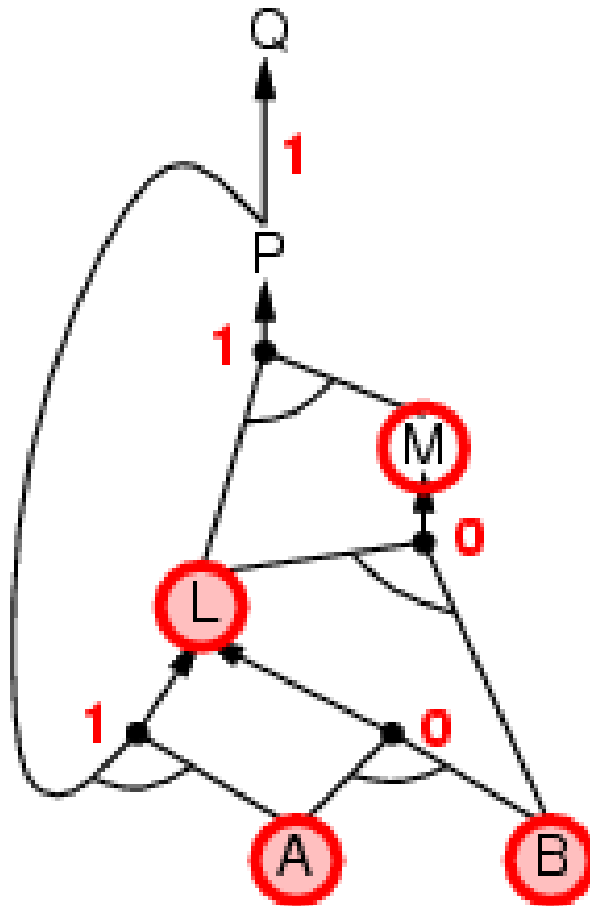
A

B



Forward chaining example

$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B



Forward chaining example

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

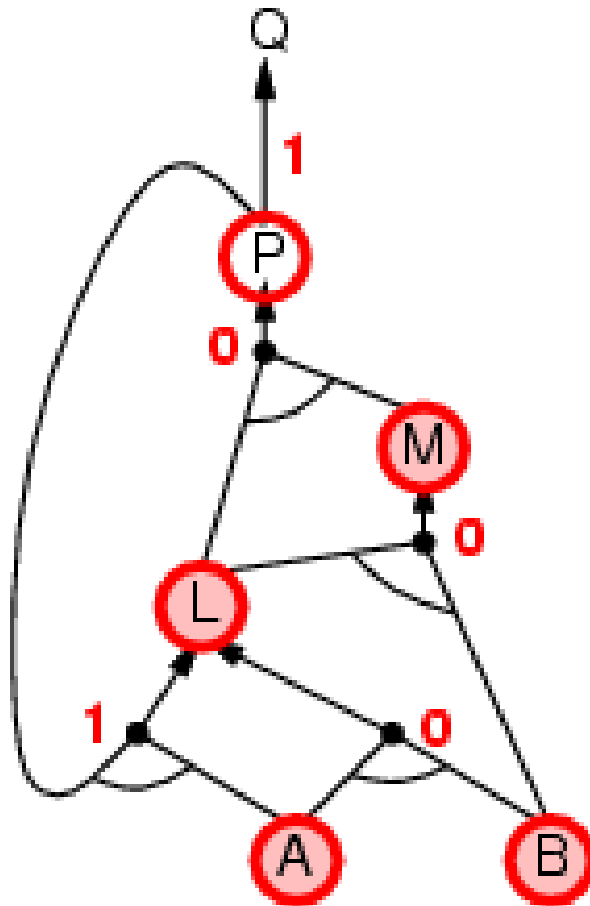
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Forward chaining example

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

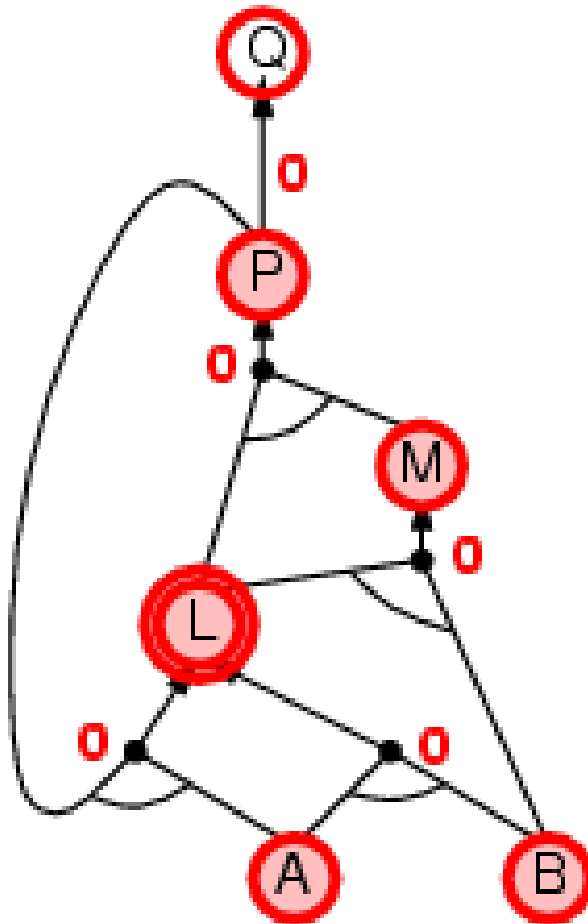
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

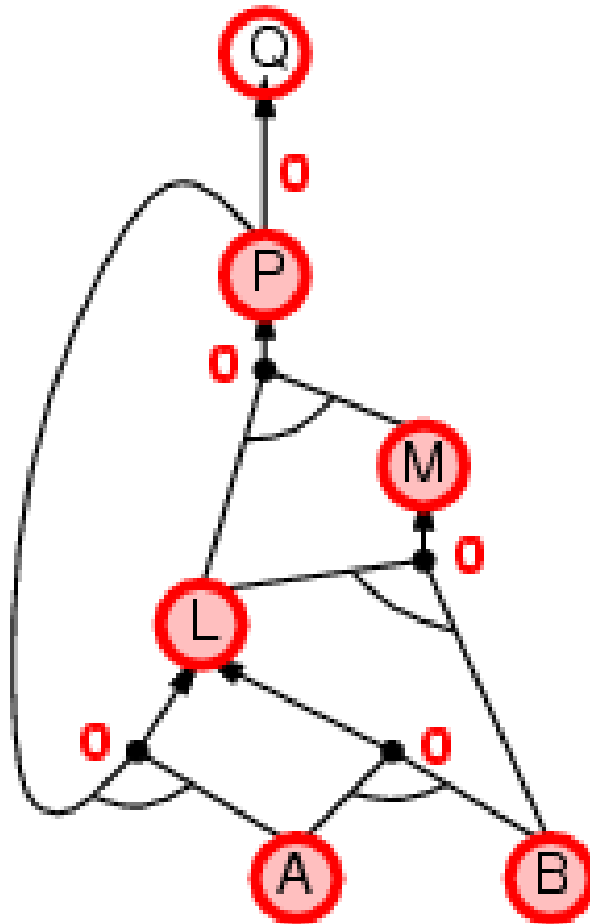
A

B



Forward chaining example

$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B



Forward chaining example

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

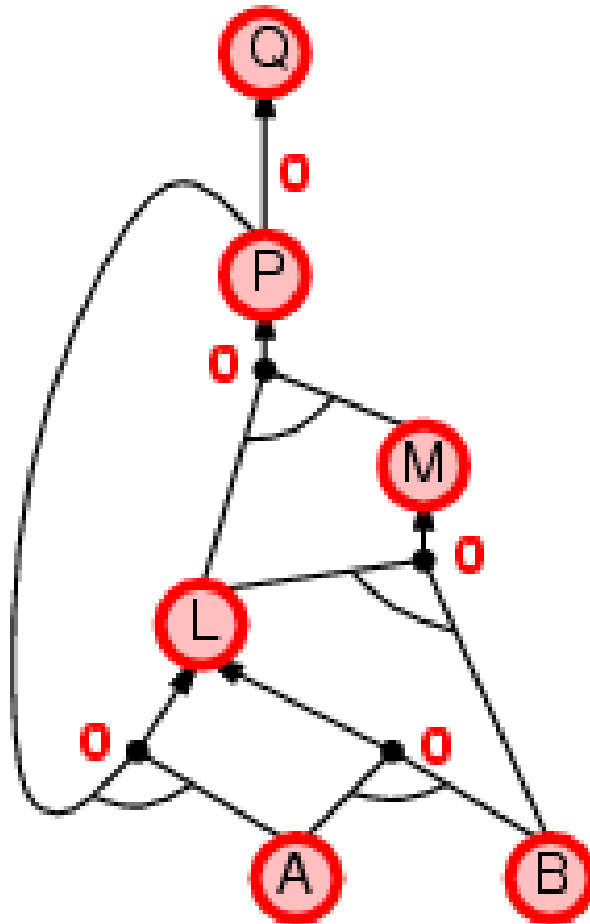
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B



Proof of completeness

- FC derives every atomic sentence that is entailed by KB
 1. FC reaches a **fixed point** where no new atomic sentences are derived
 2. Consider the final state as a model m , assigning true/false to symbols
 3. Every clause in the original KB is true in m
$$\mathbf{a_1 \wedge \dots \wedge a_k \Rightarrow b}$$
 4. Hence m is a model of KB
 5. If $KB \models q$, q is true in **every** model of KB , including m

Backward chaining

Idea: work backwards from the query q :

to prove q by BC,

check if q is known already, or

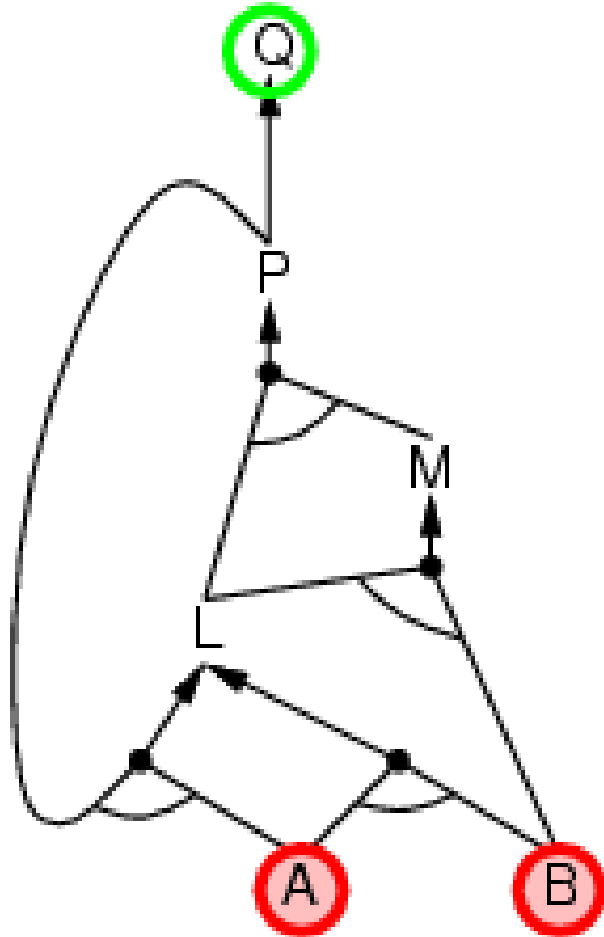
prove by BC all premises of some rule concluding q

Avoid loops: check if new subgoal is already on the goal stack

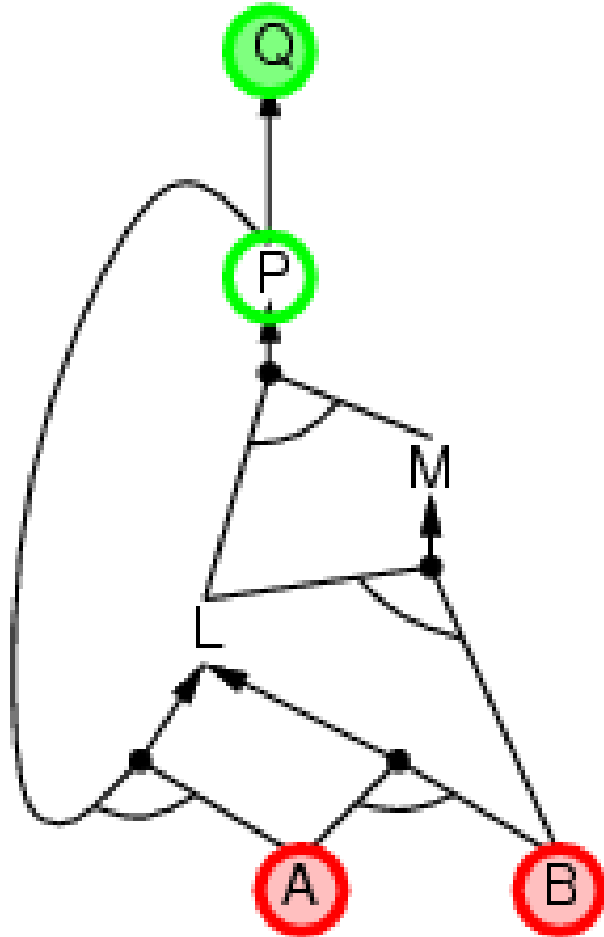
Avoid repeated work: check if new subgoal

1. has already been proved true, or
2. has already failed

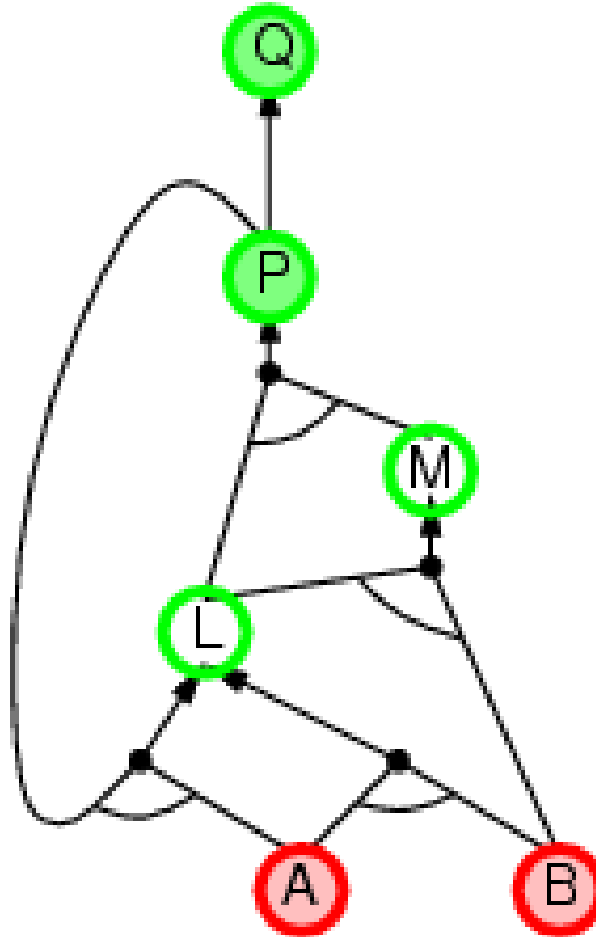
Backward chaining example



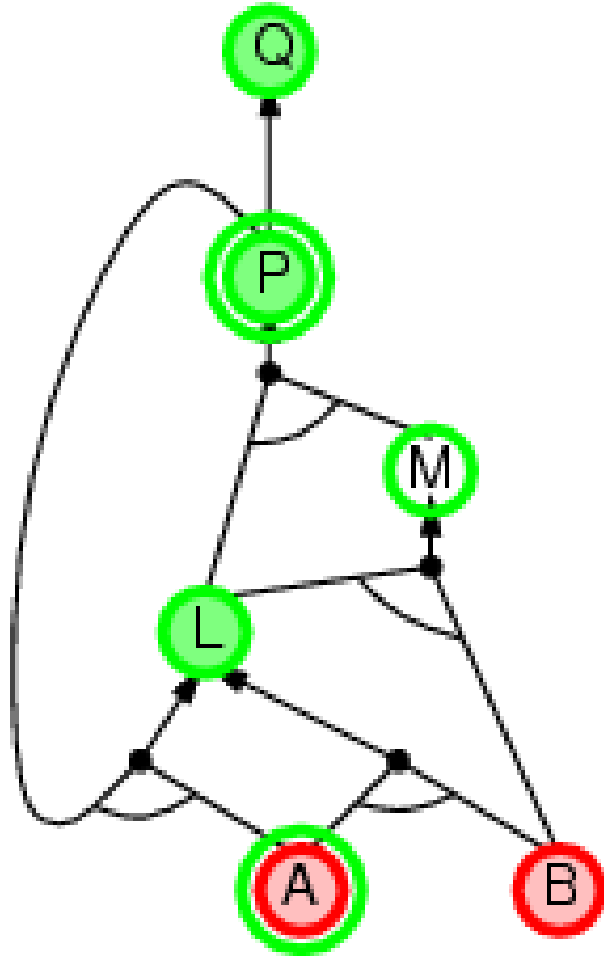
Backward chaining example



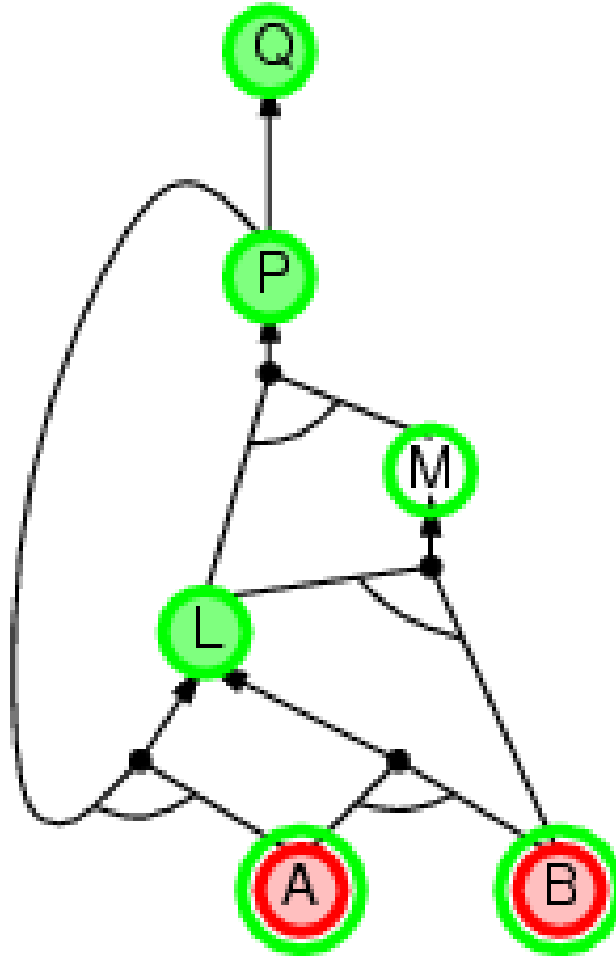
Backward chaining example



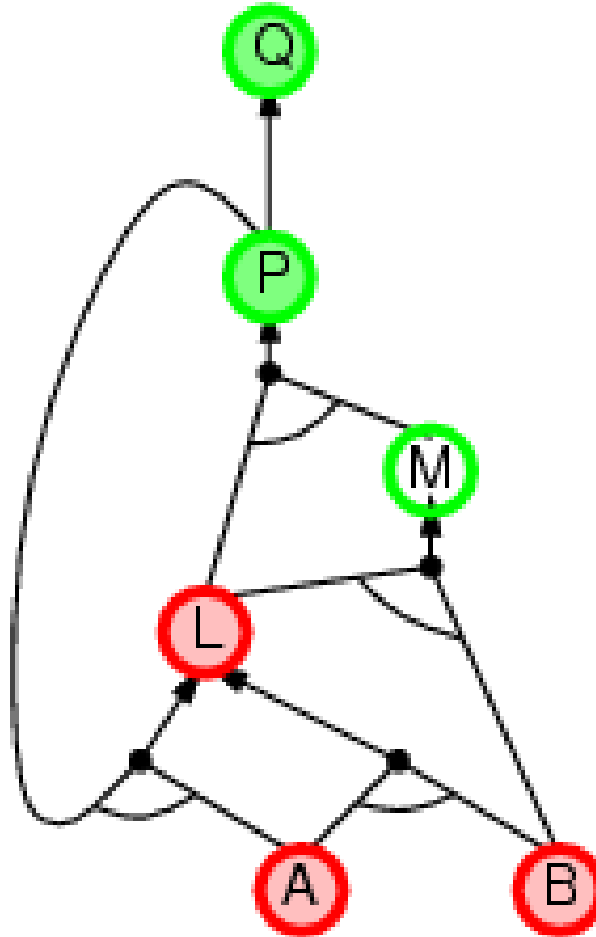
Backward chaining example



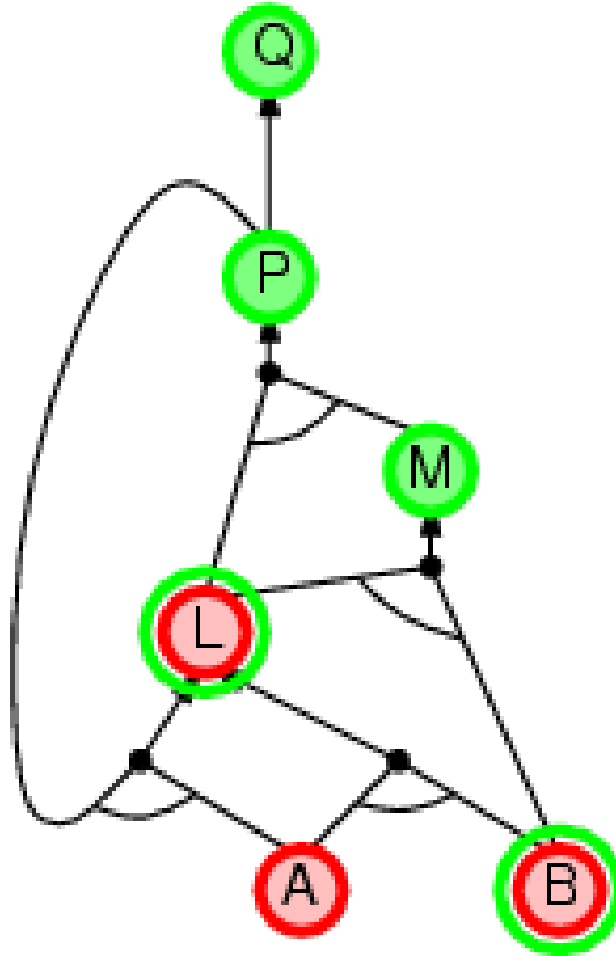
Backward chaining example



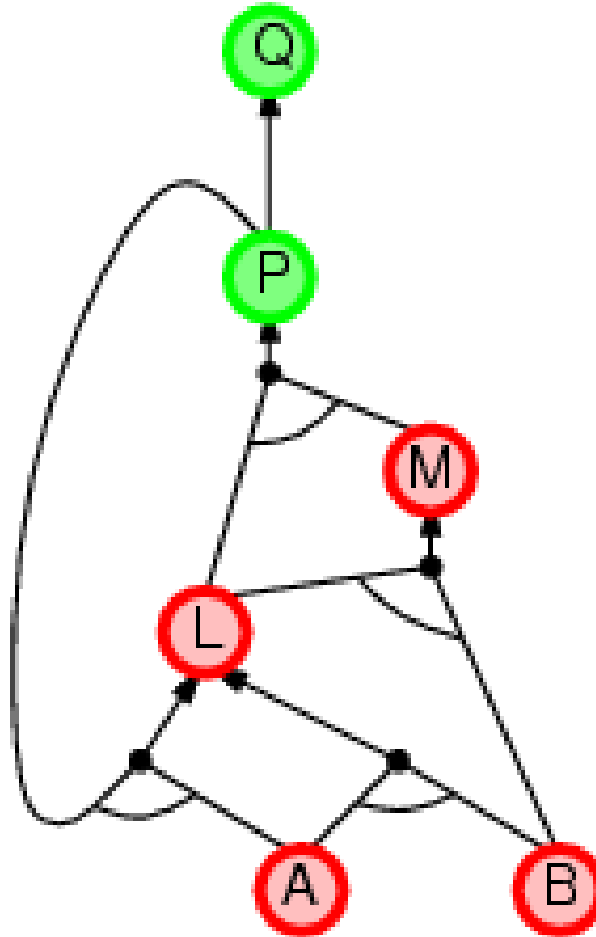
Backward chaining example



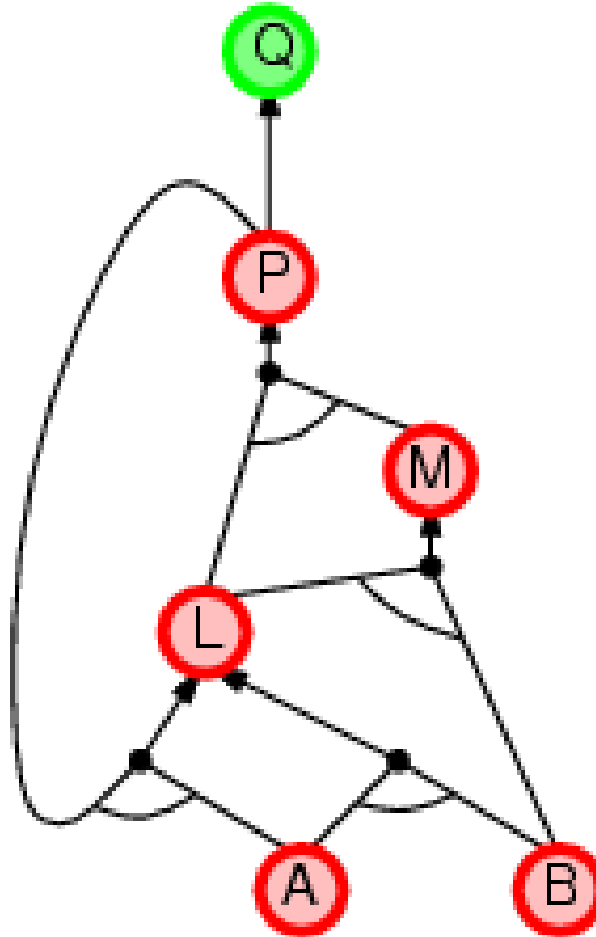
Backward chaining example



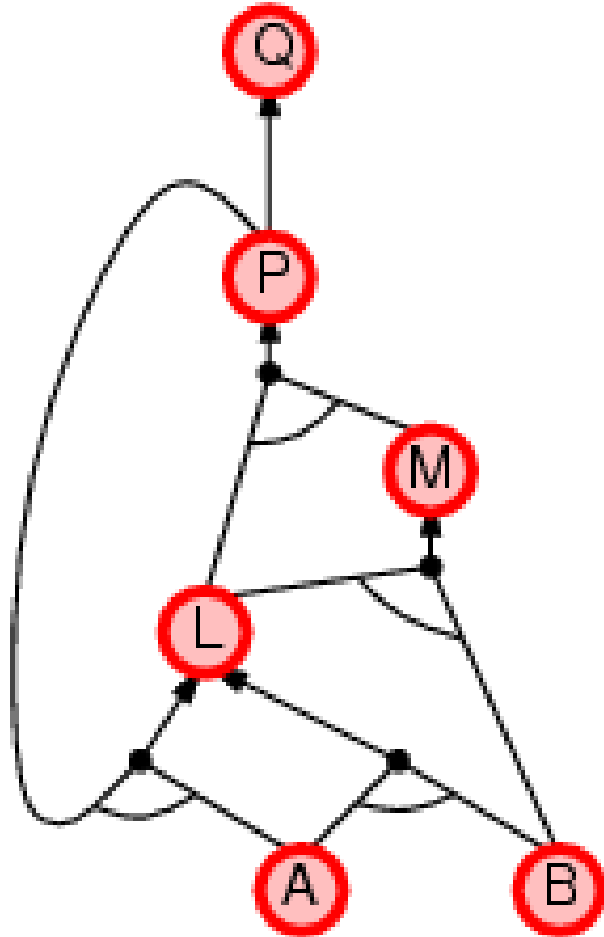
Backward chaining example



Backward chaining example



Backward chaining example



Forward vs. backward chaining

- FC is **data-driven**, automatic, unconscious processing,
 - e.g., object recognition, routine decisions
- FC may do lots of work that is irrelevant to the goal
- BC is **goal-driven**, appropriate for problem-solving,
 - e.g., Where are my keys? How do I get into a PhD program?
Where is home? What is the result of equation x ?
- Complexity of BC can be **much less** than linear in size of KB

Efficient propositional inference

Two families of efficient algorithms for propositional inference:

Complete backtracking search algorithms

- DPLL algorithm (Davis, Putnam, Logemann, Loveland)
- Incomplete local search algorithms
 - WalkSAT algorithm

The DPLL algorithm

Determine if an input propositional logic sentence (in CNF) is satisfiable.

Improvements over truth table enumeration:

1. Early termination
A clause is true if any literal is true.
A sentence is false if any clause is false.
2. Pure symbol heuristic
Pure symbol: always appears with the same "sign" in all clauses.
e.g., In the three clauses $(A \vee \neg B)$, $(\neg B \vee \neg C)$, $(\neg C \vee A)$, A and B are pure, C is impure.
Make a pure symbol literal true.
3. Unit clause heuristic
Unit clause: only one literal in the clause
The only literal in a unit clause must be true.

The DPLL algorithm

function DPLL-SATISFIABLE?(*s*) **returns** *true* or *false*

inputs: *s*, a sentence in propositional logic

clauses $\leftarrow$ the set of clauses in the CNF representation of *s*

symbols $\leftarrow$ a list of the proposition symbols in *s*

return DPLL(*clauses*, *symbols*, [])

function DPLL(*clauses*, *symbols*, *model*) **returns** *true* or *false*

if every clause in *clauses* is true in *model* **then return** *true*

if some clause in *clauses* is false in *model* **then return** *false*

P, *value* $\leftarrow$ FIND-PURE-SYMBOL(*symbols*, *clauses*, *model*)

if *P* is non-null **then return** DPLL(*clauses*, *symbols* - *P*, [*P* = *value* | *model*])

P, *value* $\leftarrow$ FIND-UNIT-CLAUSE(*clauses*, *model*)

if *P* is non-null **then return** DPLL(*clauses*, *symbols* - *P*, [*P* = *value* | *model*])

P $\leftarrow$ FIRST(*symbols*); *rest* $\leftarrow$ REST(*symbols*)

return DPLL(*clauses*, *rest*, [*P* = *true* | *model*]) **or**

DPLL(*clauses*, *rest*, [*P* = *false* | *model*])

The WalkSAT algorithm

- Incomplete, local search algorithm
- Evaluation function: The min-conflict heuristic of minimizing the number of unsatisfied clauses
- Balance between greediness and randomness

The WalkSAT algorithm

```
function WALKSAT(clauses, p, max-flips) returns a satisfying model or failure
  inputs: clauses, a set of clauses in propositional logic
         p, the probability of choosing to do a “random walk” move
         max-flips, number of flips allowed before giving up
  model ← a random assignment of true/false to the symbols in clauses
  for i = 1 to max-flips do
    if model satisfies clauses then return model
    clause ← a randomly selected clause from clauses that is false in model
    with probability p flip the value in model of a randomly selected symbol
      from clause
    else flip whichever symbol in clause maximizes the number of satisfied clauses
  return failure
```

Hard satisfiability problems

- Consider random 3-CNF sentences. e.g.,

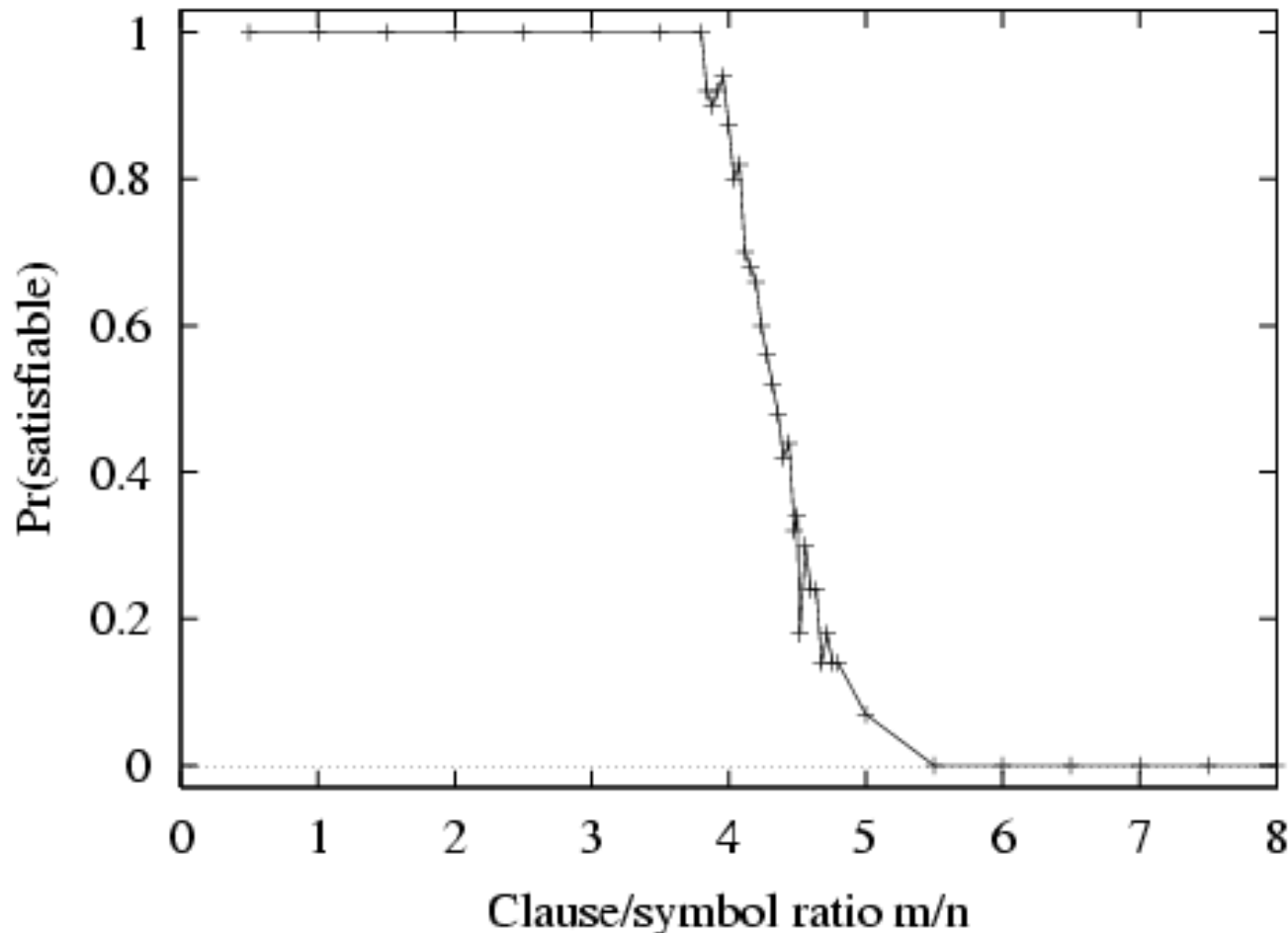
$$(\neg D \vee \neg B \vee C) \wedge (B \vee \neg A \vee \neg C) \wedge (\neg C \vee \neg B \vee E) \wedge (E \vee \neg D \vee B) \wedge (B \vee E \vee \neg C)$$

m = number of clauses

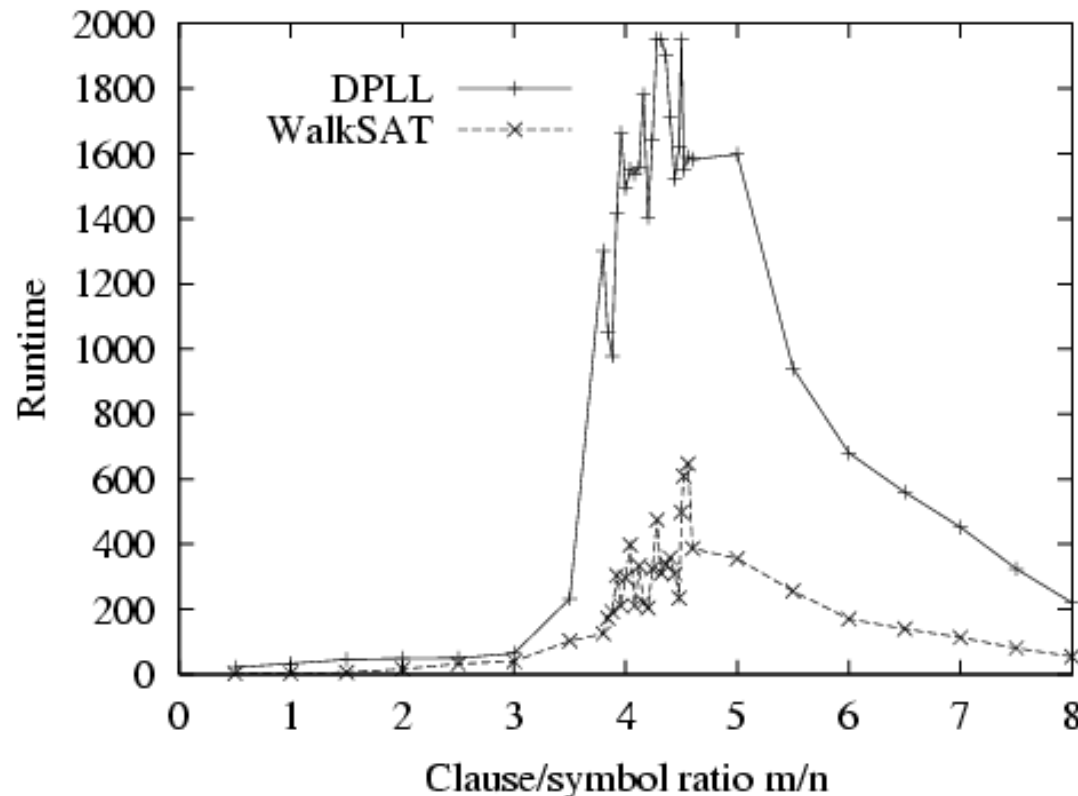
n = number of symbols

- Hard problems seem to cluster near $m/n = 4.3$ (critical point)

Hard satisfiability problems



Hard satisfiability problems



- Median runtime for 100 **satisfiable** random 3-CNF sentences, $n = 50$

Inference-based agents in the wumpus world

A wumpus-world agent using propositional logic:

$$\begin{aligned}& \neg P_{1,1} \\& \neg W_{1,1} \\& B_{x,y} \Leftrightarrow (P_{x,y+1} \vee P_{x,y-1} \vee P_{x+1,y} \vee P_{x-1,y}) \\& S_{x,y} \Leftrightarrow (W_{x,y+1} \vee W_{x,y-1} \vee W_{x+1,y} \vee W_{x-1,y}) \\& W_{1,1} \vee W_{1,2} \vee \dots \vee W_{4,4} \\& \neg W_{1,1} \vee \neg W_{1,2} \\& \neg W_{1,1} \vee \neg W_{1,3} \\& \dots\end{aligned}$$

$\Rightarrow$ 64 distinct proposition symbols, 155 sentences

function PL-WUMPUS-AGENT(*percept*) **returns** an *action*

inputs: *percept*, a list, [*stench*, *breeze*, *glitter*]

static: *KB*, initially containing the “physics” of the wumpus world
x, y, orientation, the agent’s position (init. [1,1]) and orient. (init. *right*)
visited, an array indicating which squares have been visited, initially *false*
action, the agent’s most recent action, initially null
plan, an action sequence, initially empty

update *x, y, orientation, visited* based on *action*

if *stench* **then** TELL(*KB*, $S_{x,y}$) **else** TELL(*KB*, $\neg S_{x,y}$)

if *breeze* **then** TELL(*KB*, $B_{x,y}$) **else** TELL(*KB*, $\neg B_{x,y}$)

if *glitter* **then** *action* $\leftarrow$ *grab*

else if *plan* is nonempty **then** *action* $\leftarrow$ POP(*plan*)

else if for some fringe square $[i,j]$, ASK(*KB*, $(\neg P_{i,j} \wedge \neg W_{i,j})$) is *true* or
for some fringe square $[i,j]$, ASK(*KB*, $(P_{i,j} \vee W_{i,j})$) is *false* **then do**
plan $\leftarrow$ A*-GRAPH-SEARCH(ROUTE-PB($[x,y]$, *orientation*, $[i,j]$, *visited*))
action $\leftarrow$ POP(*plan*)

else *action* $\leftarrow$ a randomly chosen move

return *action*

Summary

- Logical agents apply **inference** to a **knowledge base** to derive new information and make decisions
- Basic concepts of logic:
 - **syntax**: formal structure of **sentences**
 - **semantics**: **truth** of sentences wrt **models**
 - **entailment**: necessary truth of one sentence given another
 - **inference**: deriving sentences from other sentences
 - **soundness**: derivations produce only entailed sentences
 - **completeness**: derivations can produce all entailed sentences
- Wumpus world requires the ability to represent partial and negated information, reason by cases, etc.
- Resolution is complete for propositional logic
Forward, backward chaining are linear-time, complete for Horn clauses
- Propositional logic lacks expressive power



University of Colorado
Colorado Springs



University of Colorado

Boulder | Colorado Springs | Denver | Anschutz Medical Campus

Lecture 8: First Order Logic Part III

Russell and Norvig: Chapter 8

CS-4820/5820

Tu/Th 12:15 PM-1:30 PM

Room: Centennial Hall 106

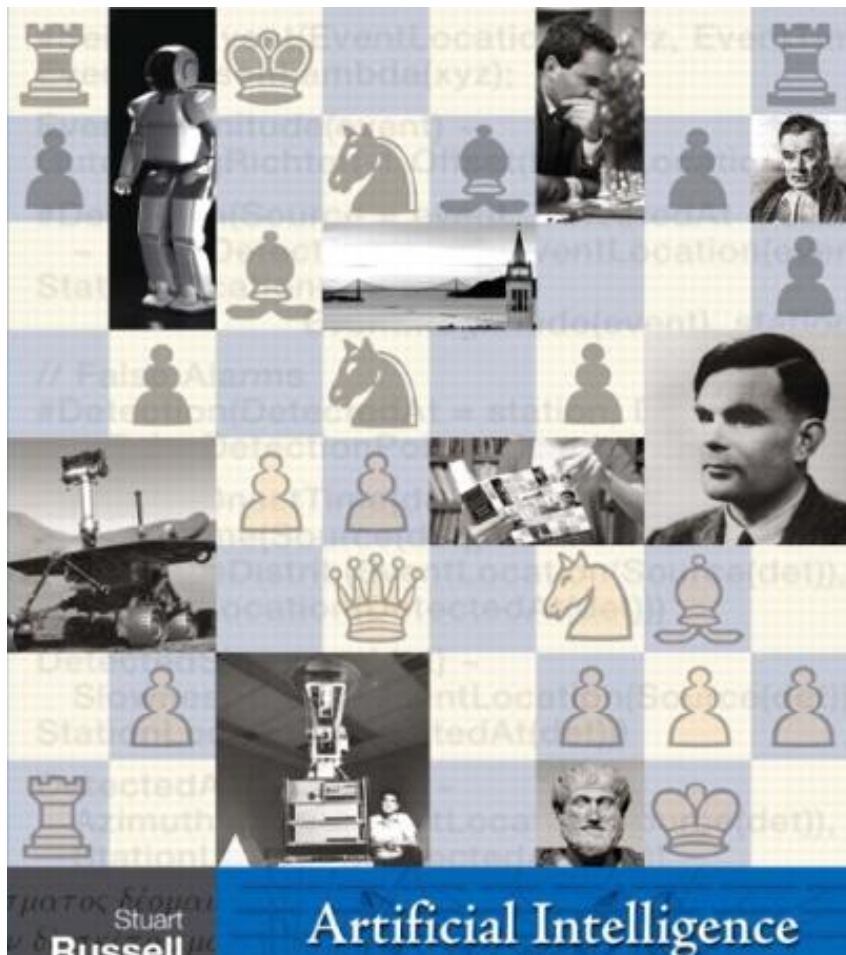
Instructor: Adham Atyabi

Office: Engineering 243

Office Hours: Mon 9:00 AM-14:00 PM.

Email: aatyabi@uccs.edu

**Teaching Assistant: Ali Al Shami
(aalshami@uccs.edu)**



Outline

- Why FOL?
- Syntax and semantics of FOL
- Using FOL
- Wumpus world in FOL
- Knowledge engineering in FOL

Pros and cons of propositional logic

- ☺ Propositional logic is **declarative**
- ☺ Propositional logic allows partial/disjunctive/negated information
 - (unlike most data structures and databases)
- ☺ Propositional logic is **compositional**:
 - meaning of $B_{1,1} \wedge P_{1,2}$ is derived from meaning of $B_{1,1}$ and of $P_{1,2}$
- ☺ Meaning in propositional logic is **context-independent**
 - (unlike natural language, where meaning depends on context)
- ☹ Propositional logic has very limited expressive power
 - (unlike natural language)
 - E.g., cannot say "pits cause breezes in adjacent squares"
 - ***except by writing one sentence for each square***

First-order logic

- propositional logic assumes the world contains **facts**,
- first-order logic (like natural language) assumes the world contains
 - **Objects**: people, houses, numbers, colors, baseball games, wars, ...
 - **Relations**: red, round, prime, brother of, bigger than, part of, comes between, ...
 - **Functions**: father of, best friend, one more than, plus, ...

Syntax of FOL: Basic elements

- Constants KingJohn, 2, NUS,...
- Predicates Brother, >,...
- Functions Sqrt, LeftLegOf,...
- Variables $x, y, a, b, \dots$
- Connectives $\neg, \Rightarrow, \wedge, \vee, \Leftrightarrow$
- Equality $=$
- Quantifiers $\forall, \exists$

Atomic sentences

Atomic sentence = *predicate* ($term_1, \dots, term_n$)
or $term_1 = term_2$

Term = *function* ($term_1, \dots, term_n$)
or *constant* or *variable*

- E.g., *Brother(KingJohn, RichardTheLionheart)*
(Length(LeftLegOf(Richard)),
Length(LeftLegOf(KingJohn))

Complex sentences

- Complex sentences are made from atomic sentences using connectives

$$\neg S, S_1 \wedge S_2, S_1 \vee S_2, S_1 \Rightarrow S_2, S_1 \Leftrightarrow S_2,$$

E.g. *Sibling(KingJohn, Richard) $\Rightarrow$ Sibling(Richard, KingJohn)*

$$(1,2) \vee \leq (1,2)$$

$$(1,2) \wedge \neg > (1,2)$$

Or

Brother (R, J) $\wedge$ brother(J, R) $\vee$ $\neg$ King(Richard) $\Rightarrow$ king(john)

Truth in first-order logic

- Sentences are true with respect to a **model** and an **interpretation**
- Model contains objects (**domain elements**) and relations among them
- Interpretation specifies referents for
 - constant symbols** → **objects**
 - predicate symbols** → **relations**
 - function symbols** → **functional relations**
- An atomic sentence $predicate(term_1, \dots, term_n)$ is true iff the **objects** referred to by $term_1, \dots, term_n$ are in the **relation** referred to by $predicate$

Interpretation

An interpretation I is defined by a **mapping** to the **domain of discourse D** or **relations on D**

- **domain of discourse:** a set of objects in the world we represent and refer to;

An interpretation I maps:

- Constant symbols to objects in D

$$I(\text{John}) = \text{stick figure}$$

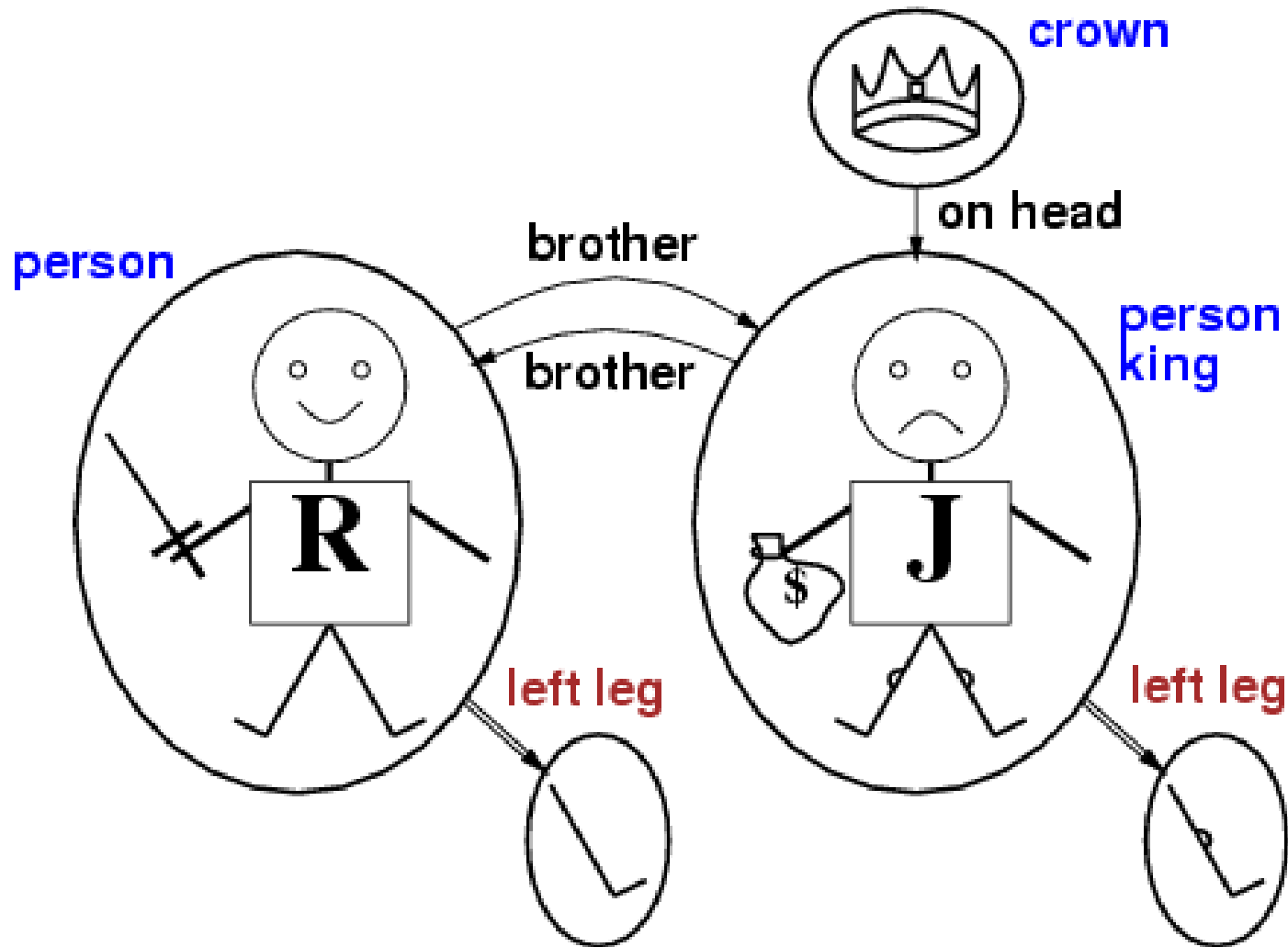
- Predicate symbols to relations, properties on D

$$I(\text{brother}) = \left\{ \langle \text{stick figure}, \text{robot} \rangle; \langle \text{stick figure}, \text{stick figure} \rangle; \dots \right\}$$

- Function symbols to functional relations on D

$$I(\text{father-of}) = \left\{ \langle \text{stick figure} \rangle \rightarrow \text{stick figure}; \langle \text{stick figure} \rangle \rightarrow \text{robot}; \dots \right\}$$

Models for FOL: Example



Universal quantification

- $\forall \langle \text{variables} \rangle \langle \text{sentence} \rangle$

Everyone at UCCS is smart: $\forall x \text{ At}(x, \text{UCCS}) \Rightarrow \text{Smart}(x)$

- $\forall x P$ is true in a model m if P is true with x being each possible object in the model
- Roughly speaking, equivalent to the **conjunction** of **instantiations** of P
 - $\text{At}(\text{KingJohn}, \text{UCCS}) \Rightarrow \text{Smart}(\text{KingJohn})$
 - $\wedge \text{At}(\text{Richard}, \text{UCCS}) \Rightarrow \text{Smart}(\text{Richard})$
 - $\wedge \text{At}(\text{UCCS}, \text{UCCS}) \Rightarrow \text{Smart}(\text{UCCS})$
 - $\wedge \dots$

A common mistake to avoid

- Typically, $\Rightarrow$ is the main connective with $\forall$

- Common mistake: using $\wedge$ as the main connective with $\forall$:

$$\forall x \text{ At}(x, \text{UCCS}) \wedge \text{Smart}(x)$$

means “Everyone is at UCCS and everyone is smart”

Existential quantification

- $\exists \langle \text{variables} \rangle \langle \text{sentence} \rangle$
- Someone at UCCS is smart:
 $\exists x \text{ At}(x, \text{UCCS}) \wedge \text{Smart}(x)$
- $\exists x P$ is true in a model m iff P is true with x being some possible object in the model
- Roughly speaking, equivalent to the **disjunction** of **instantiations** of P
 - $\text{At}(\text{KingJohn}, \text{UCCS}) \wedge \text{Smart}(\text{KingJohn})$
 - $\vee \text{ At}(\text{Richard}, \text{UCCS}) \wedge \text{Smart}(\text{Richard})$
 - $\vee \text{ At}(\text{UCCS}, \text{UCCS}) \wedge \text{Smart}(\text{UCCS})$
 - $\vee \dots$

Another common mistake to avoid

- Typically, $\wedge$ is the main connective with $\exists$
- Common mistake: using $\Rightarrow$ as the main connective with $\exists$:

$$\exists x \text{ At}(x, \text{UCCS}) \Rightarrow \text{Smart}(x)$$

is true if there is anyone who is not at UCCS!

Properties of quantifiers

- $\forall x \forall y$ is the same as $\forall y \forall x$
- $\exists x \exists y$ is the same as $\exists y \exists x$
- $\exists x \forall y$ is **not** the same as $\forall y \exists x$
- $\exists x \forall y \text{ Loves}(x,y)$
 - “There is a person who loves everyone in the world”
- $\forall y \exists x \text{ Loves}(x,y)$
 - “Everyone in the world is loved by at least one person”
- **Quantifier duality**: each can be expressed using the other
- $\forall x \text{ Likes}(x, \text{IceCream}) \quad \neg \exists x \neg \text{Likes}(x, \text{IceCream})$
- $\exists x \text{ Likes}(x, \text{Broccoli}) \quad \neg \forall x \neg \text{Likes}(x, \text{Broccoli})$

Equality

- $term_1 = term_2$ is true under a given interpretation if and only if $term_1$ and $term_2$ refer to the same object
- E.g., definition of *Sibling* in terms of *Parent*:
$$\forall x, y \text{ Sibling}(x, y) \Leftrightarrow [\neg(x = y) \wedge \exists m, f \neg (m = f) \wedge \text{Parent}(m, x) \wedge \text{Parent}(f, x) \wedge \text{Parent}(m, y) \wedge \text{Parent}(f, y)]$$

Using FOL

The kinship domain:

- Brothers are siblings

$$\forall x,y \text{ Brother}(x,y) \Leftrightarrow \text{Sibling}(x,y)$$

- One's mother is one's female parent

$$\forall m,c \text{ Mother}(c) = m \Leftrightarrow (\text{Female}(m) \wedge \text{Parent}(m,c))$$

- “Sibling” is symmetric

$$\forall x,y \text{ Sibling}(x,y) \Leftrightarrow \text{Sibling}(y,x)$$

Using FOL

The set domain:

1. $\forall s \text{ Set}(s) \Leftrightarrow (s = \{\}) \vee (\exists x, s_2 \text{ Set}(s_2) \wedge s = \{x|s_2\})$
2. $\neg \exists x, s \{x|s\} = \{\}$
3. $\forall x, s x \in s \Leftrightarrow s = \{x|s\}$
4. $\forall x, s x \in s \Leftrightarrow [\exists y, s_2 \{ (s = \{y|s_2\} \wedge (x = y \vee x \in s_2))]$
5. $\forall s_1, s_2 s_1 \subseteq s_2 \Leftrightarrow (\forall x x \in s_1 \Rightarrow x \in s_2)$
6. $\forall s_1, s_2 (s_1 = s_2) \Leftrightarrow (s_1 \subseteq s_2 \wedge s_2 \subseteq s_1)$
7. $\forall x, s_1, s_2 x \in (s_1 \cap s_2) \Leftrightarrow (x \in s_1 \wedge x \in s_2)$
8. $\forall x, s_1, s_2 x \in (s_1 \cup s_2) \Leftrightarrow (x \in s_1 \vee x \in s_2)$

Interacting with FOL KBs

- Suppose a wumpus-world agent is using an FOL KB and perceives a smell and a breeze (but no glitter) at $t=5$:

Tell(KB, Percept([Smell, Breeze, None], 5))

Ask(KB, $\exists a$ BestAction($a, 5$)) returns true or false

i.e., does the KB entail some best action at $t=5$?

ASKVARS($\exists a$ BestAction($a, 5$)) returns a respond, like {a/left, a/down, a/Shoot}

Answer: *Yes*, {a/Shoot} $\leftarrow$ **substitution** (binding list)

- Given a sentence S and a substitution σ ,
- $S\sigma$ denotes the result of plugging σ into S ; e.g.,
 $S = \text{Smarter}(x, y)$
 $\sigma = \{x/\text{Hillary}, y/\text{Bill}\}$
 $S\sigma = \text{Smarter}(\text{Hillary}, \text{Bill})$
- Ask(KB, S) returns some/all σ such that $\text{KB} \models \sigma$

Knowledge base for the wumpus world

- Perception

- $\forall t, s, b \text{ Percept}([s, b, \text{Glitter}], t) \Rightarrow \text{Glitter}(t)$

- Reflex

- $\forall t \text{ Glitter}(t) \Rightarrow \text{BestAction}(\text{Grab}, t)$

Deducing hidden properties

- $\forall x,y,a,b \text{ Adjacent}([x,y],[a,b]) \Leftrightarrow$
 $[a,b] \in \{[x+1,y], [x-1,y],[x,y+1],[x,y-1]\}$

Properties of squares:

- $\forall s,t \text{ At}(\text{Agent},s,t) \wedge \text{Breeze}(t) \Rightarrow \text{Breezy}(s)$

Squares are breezy near a pit:

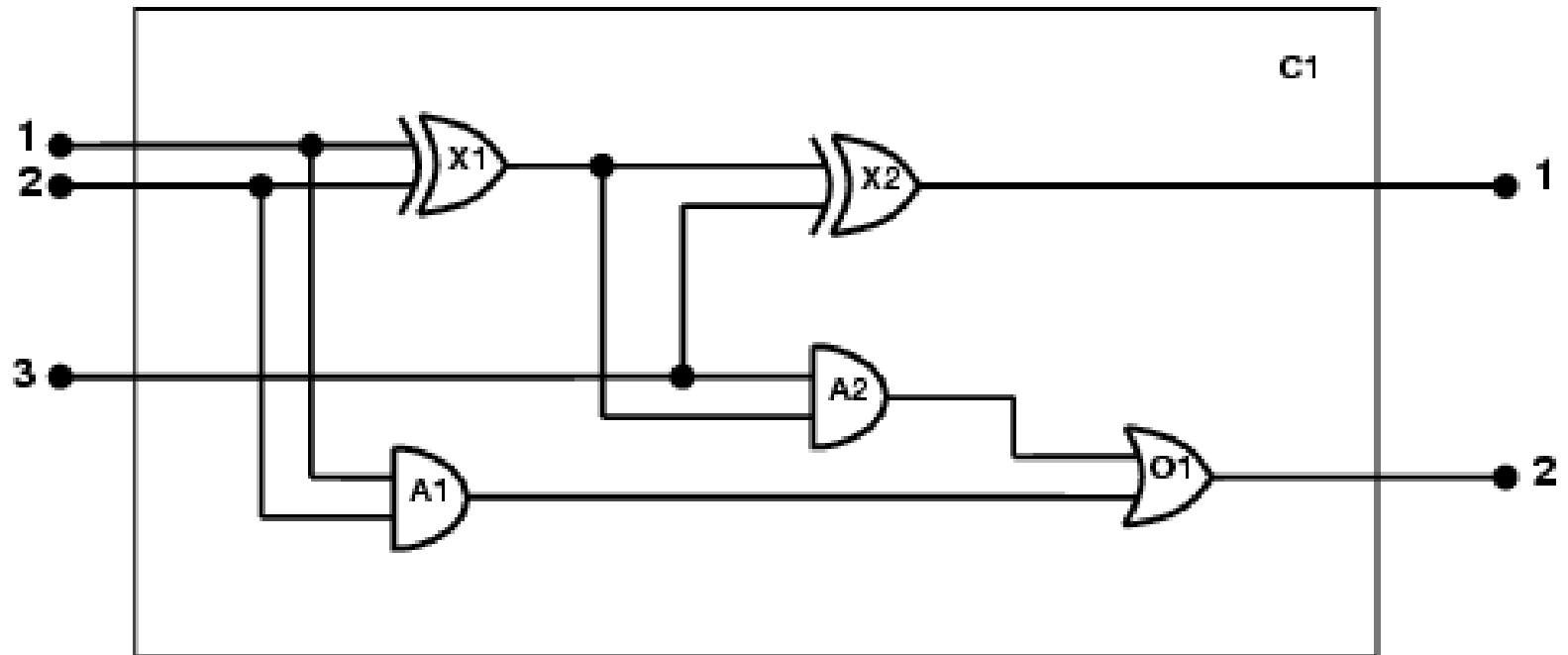
- **Diagnostic** rule---infer cause from effect
 $\forall s \text{ Breezy}(s) \Rightarrow \exists r \text{ Adjacent}(r,s) \wedge \text{Pit}(r)$
- **Causal** rule---infer effect from cause
 $\forall r \text{ Pit}(r) \Rightarrow [\forall s \text{ Adjacent}(r,s) \Rightarrow \text{Breezy}(s)]$

Knowledge engineering in FOL

1. Identify the task
2. Assemble the relevant knowledge
3. Decide on a vocabulary of predicates, functions, and constants
4. Encode general knowledge about the domain
5. Encode a description of the specific problem instance
6. Pose queries to the inference procedure and get answers
7. Debug the knowledge base

The electronic circuits domain

One-bit full adder



The electronic circuits domain

1. Identify the task

- Does the circuit actually add properly? (circuit verification)

2. Assemble the relevant knowledge

- Composed of wires and gates; Types of gates (AND, OR, XOR, NOT)
- Irrelevant: size, shape, color, cost of gates

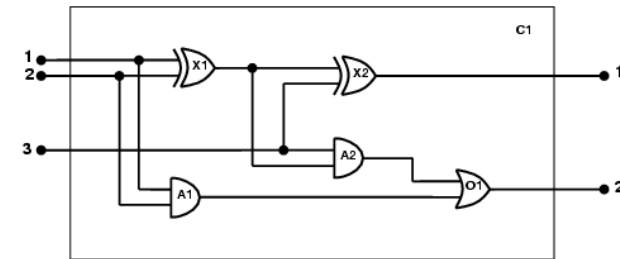
3. Decide on a vocabulary

- Alternatives:
 $Type(X_1) = XOR$
 $Type(X_1, XOR)$
 $XOR(X_1)$

The electronic circuits domain

4. Encode general knowledge of the domain

- $\forall t_1, t_2 \text{ Connected}(t_1, t_2) \Rightarrow \text{Signal}(t_1) = \text{Signal}(t_2)$
- $\forall t \text{ Signal}(t) = 1 \vee \text{Signal}(t) = 0$
- $1 \neq 0$
- $\forall t_1, t_2 \text{ Connected}(t_1, t_2) \Rightarrow \text{Connected}(t_2, t_1)$
- $\forall g \text{ Type}(g) = \text{OR} \Rightarrow \text{Signal}(\mathbf{Out}(1, g)) = 1 \Leftrightarrow \exists n \text{ Signal}(\mathbf{In}(n, g)) = 1$
- $\forall g \text{ Type}(g) = \text{AND} \Rightarrow \text{Signal}(\mathbf{Out}(1, g)) = 0 \Leftrightarrow \exists n \text{ Signal}(\mathbf{In}(n, g)) = 0$
- $\forall g \text{ Type}(g) = \text{XOR} \Rightarrow \text{Signal}(\mathbf{Out}(1, g)) = 1 \Leftrightarrow \text{Signal}(\mathbf{In}(1, g)) \neq \text{Signal}(\mathbf{In}(2, g))$
- $\forall g \text{ Type}(g) = \text{NOT} \Rightarrow \text{Signal}(\mathbf{Out}(1, g)) \neq \text{Signal}(\mathbf{In}(1, g))$

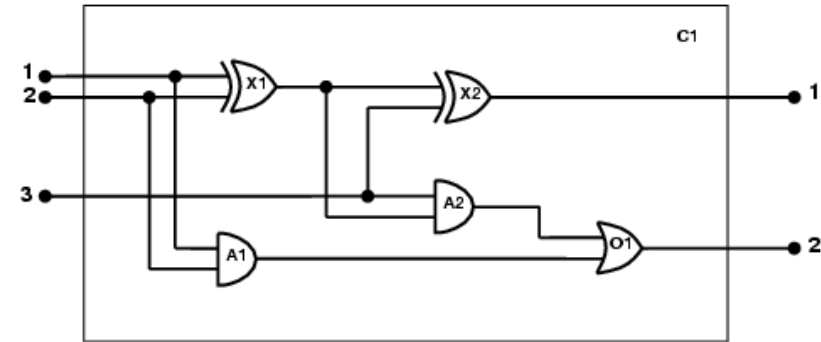


The electronic circuits domain

5. Encode the specific problem instance

Type(X_1) = XOR Type(X_2) = XOR
Type(A_1) = AND Type(A_2) = AND
Type(O_1) = OR

Connected(Out(1, X_1),In(1, X_2))
Connected(Out(1, X_1),In(2, A_2))
Connected(Out(1, A_2),In(1, O_1))
Connected(Out(1, A_1),In(2, O_1))
Connected(Out(1, X_2),Out(1, C_1))
Connected(Out(1, O_1),Out(2, C_1))



Connected(In(1, C_1),In(1, X_1))
Connected(In(1, C_1),In(1, A_1))
Connected(In(2, C_1),In(2, X_1))
Connected(In(2, C_1),In(2, A_1))
Connected(In(3, C_1),In(2, X_2))
Connected(In(3, C_1),In(1, A_2))

The electronic circuits domain

6. Pose queries to the inference procedure

What are the possible sets of values of all the terminals for the adder circuit?

$$\begin{aligned} \exists i_1, i_2, i_3, o_1, o_2 \text{ Signal(In}(1, C_1)) = i_1 \wedge \text{Signal(In}(2, C_1)) = i_2 \wedge \\ \text{Signal(In}(3, C_1)) = i_3 \wedge \text{Signal(Out}(1, C_1)) = o_1 \wedge \\ \text{Signal(Out}(2, C_1)) = o_2 \end{aligned}$$

7. Debug the knowledge base

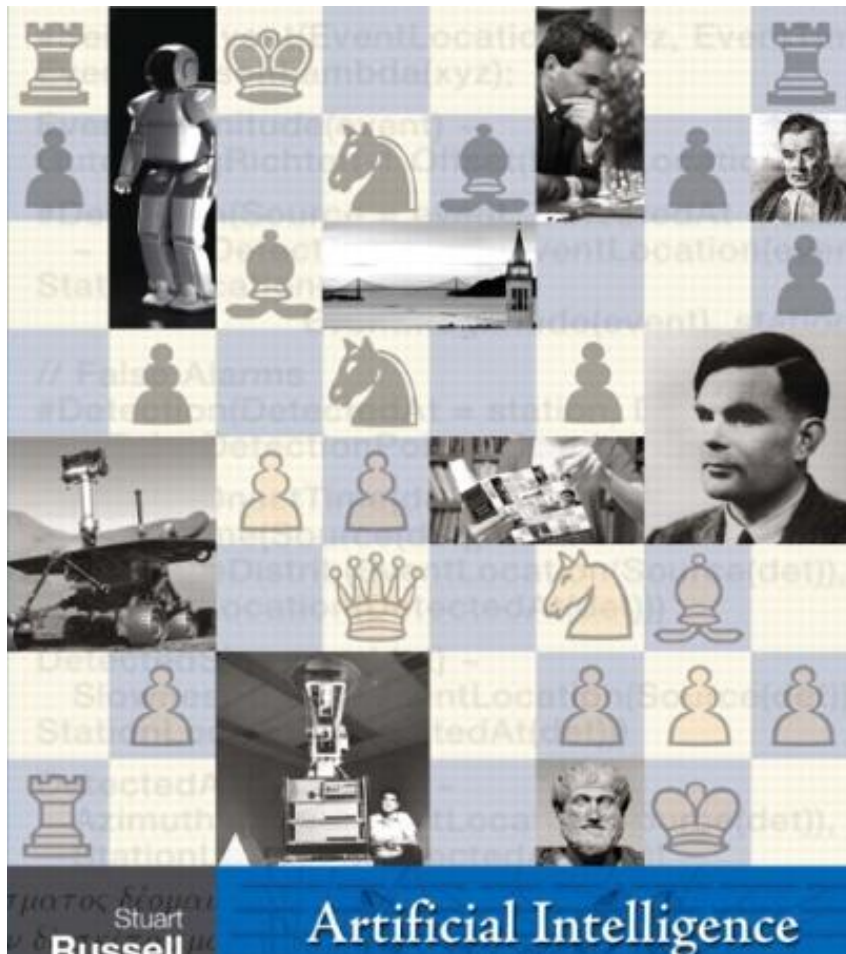
May have omitted assertions like $1 \neq 0$

Summary

- First-order logic:
 - objects and relations are semantic primitives
 - syntax: constants, functions, predicates, equality, quantifiers
- Increased expressive power: sufficient to define wumpus world

Lecture 8: Inference in first-order logic Part IV

Russell and Norvig: Chapter 9



CS-4820/5820

Tu/Th 12:15 PM-1:30 PM

Room: Centennial Hall 106

Instructor: Adham Atyabi

Office: Engineering 243

Office Hours: Mon 9:00 AM-14:00 PM.

Email: aatyabi@uccs.edu

**Teaching Assistant: Ali Al Shami
(aalshami@uccs.edu)**

Outline

- Reducing first-order inference to propositional inference
- Unification
- Generalized Modus Ponens
- Forward chaining
- Backward chaining
- Resolution

Creating a knowledge Base

- ▶ Identify the Task
- ▶ Assemble the relevant knowledge
- ▶ Decide on a vocabulary of predicates, functions and constants
- ▶ Encode general knowledge about the domain (rules)
- ▶ Encode a description of the problem
- ▶ Pose queries to the inference procedure and get answers
- ▶ Debug the KB

Universal instantiation (UI)

- Every instantiation of a universally quantified sentence is entailed by it:

$$\frac{\forall v \ \alpha}{\text{Subst}(\{v/g\}, \alpha)}$$

for any variable v and ground term g

- E.g., lets say we have this rule,
 $\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$ so we can infer followings:

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$

$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$

$\text{King}(\text{Father}(\text{John})) \wedge \text{Greedy}(\text{Father}(\text{John})) \Rightarrow \text{Evil}(\text{Father}(\text{John}))$

.

.

- Remember that from previous lectures, we do have algorithms to solve $\text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$. The thing we don't have a way to solve right now is how to deal with is $\forall x$ term.

Existential instantiation (EI)

- For any sentence α , variable v , and constant symbol k that does **not** appear elsewhere in the knowledge base:

$$\frac{\exists v \alpha}{\text{Subst}(\{v/k\}, \alpha)}$$

- E.g., $\exists x \text{Crown}(x) \wedge \text{OnHead}(x, \text{John})$ yields:

$$\text{Crown}(C_1) \wedge \text{OnHead}(C_1, \text{John})$$

provided C_1 is a new constant symbol, called a **Skolem constant**

- Skolem constant** is a new variables that represent new inference

Reduction to propositional inference

Suppose the KB contains just the following:

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

$\text{King}(\text{John})$

$\text{Greedy}(\text{John})$

$\text{Brother}(\text{Richard}, \text{John})$

- Instantiating the universal sentence in **all possible** ways, we have:

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$

$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$

$\text{King}(\text{John})$

$\text{Greedy}(\text{John})$

$\text{Brother}(\text{Richard}, \text{John})$

- The new KB is **propositionalized**: proposition symbols are $\text{King}(\text{John})$, $\text{Greedy}(\text{John})$, $\text{Evil}(\text{John})$, $\text{King}(\text{Richard})$, etc.

Reduction contd.

- Every FOL KB can be propositionalized so as to preserve entailment
- A ground sentence is entailed by new KB if entailed by original KB
- Idea: propositionalize KB and query, apply resolution, return result
- Problem: with function symbols, there are infinitely many ground terms,
 - e.g., *Father(Father(Father(John)))*

Reduction contd.

Theorem: Herbrand (1930). If a sentence α is entailed by an FOL KB, it is entailed by a **finite** subset of the propositionalized KB

Idea: For $n = 0$ to ∞ do
 create a propositional KB by instantiating with depth- n terms
 see if α is entailed by this KB

Problem: works if α is entailed, loops if α is not entailed

Theorem: Turing (1936), Church (1936) Entailment for FOL is **semidecidable** (algorithms exist that say yes to every entailed sentence, but no algorithm exists that also says no to every nonentailed sentence.)

Problems with propositionalization

- Propositionalization seems to generate lots of irrelevant sentences.
- E.g., from:
 $\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$
 $\text{King}(\text{John})$
 $\forall y \text{ Greedy}(y)$
 $\text{Brother}(\text{Richard}, \text{John})$
- it seems obvious that *Evil(John)*, but propositionalization produces lots of facts such as *Greedy(Richard)* that are irrelevant
- With p k -array predicates and n constants, there are $p \cdot n^k$ instantiations.

Unification

- We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$

$\theta = \{x/John, y/John\}$ works

- $Unify(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$

p	q	θ
Knows(John,x)	Knows(John,Jane)	
Knows(John,x)	Knows(y,OJ)	
Knows(John,x)	Knows(y,Mother(y))	
Knows(John,x)	Knows(x,OJ)	

- Standardizing apart** eliminates overlap of variables, e.g., Knows(z_{17} , OJ)

Unification

- We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$

$\theta = \{x/John, y/John\}$ works

- $Unify(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$

p	q	θ
Knows(John,x)	Knows(John,Jane)	$\{x/Jane\}$
Knows(John,x)	Knows(y,OJ)	
Knows(John,x)	Knows(y,Mother(y))	
Knows(John,x)	Knows(x,OJ)	

- Standardizing apart** eliminates overlap of variables, e.g., $Knows(z_{17}, OJ)$

Unification

- We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$

$\theta = \{x/John, y/John\}$ works

- $Unify(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$

p	q	θ
Knows(John,x)	Knows(John,Jane)	$\{x/Jane\}$
Knows(John,x)	Knows(y,OJ)	$\{x/OJ, y/John\}$
Knows(John,x)	Knows(y,Mother(y))	
Knows(John,x)	Knows(x,OJ)	

- Standardizing apart** eliminates overlap of variables, e.g., $Knows(z_{17}, OJ)$

Unification

- We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$

$\theta = \{x/John, y/John\}$ works

- $Unify(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$

p	q	θ
Knows(John,x)	Knows(John,Jane)	$\{x/Jane\}$
Knows(John,x)	Knows(y,OJ)	$\{x/OJ, y/John\}$
Knows(John,x)	Knows(y,Mother(y))	$\{y/John, x/Mother(John)\}$
Knows(John,x)	Knows(x,OJ)	

- Standardizing apart** eliminates overlap of variables, e.g., $Knows(z_{17}, OJ)$

Unification

- We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$

$\theta = \{x/John, y/John\}$ works

- $Unify(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$

p	q	θ
Knows(John,x)	Knows(John,Jane)	$\{x/Jane\}$
Knows(John,x)	Knows(y,OJ)	$\{x/OJ, y/John\}$
Knows(John,x)	Knows(y,Mother(y))	$\{y/John, x/Mother(John)\}$
Knows(John,x)	Knows(x,OJ)	$\{fail\}$

- Standardizing apart** eliminates overlap of variables, e.g., $Knows(z_{17}, OJ)$

Unification

- To unify $Knows(John, x)$ and $Knows(y, z)$,
 $\theta = \{y/John, x/z\}$ or $\theta = \{y/John, x/John, z/John\}$
- The first unifier is **more general** than the second.
- There is a single **most general unifier** (MGU) that is unique up to renaming of variables.
MGU = $\{y/John, x/z\}$

The unification algorithm

function UNIFY(x, y, θ) **returns** a substitution to make x and y identical

inputs: x , a variable, constant, list, or compound

y , a variable, constant, list, or compound

θ , the substitution built up so far

if $\theta = \text{failure}$ **then return failure**

else if $x = y$ **then return** θ

else if VARIABLE?(x) **then return** UNIFY-VAR(x, y, θ)

else if VARIABLE?(y) **then return** UNIFY-VAR(y, x, θ)

else if COMPOUND?(x) **and** COMPOUND?(y) **then**

return UNIFY(ARGS[x], ARGS[y], UNIFY(OP[x], OP[y], θ))

else if LIST?(x) **and** LIST?(y) **then**

return UNIFY(REST[x], REST[y], UNIFY(FIRST[x], FIRST[y], θ))

else return failure

The unification algorithm

function UNIFY-VAR(var, x, θ) **returns** a substitution

inputs: var , a variable

x , any expression

θ , the substitution built up so far

if $\{var/val\} \in \theta$ **then return** UNIFY(val, x, θ)

else if $\{x/val\} \in \theta$ **then return** UNIFY(var, val, θ)

else if OCCUR-CHECK?(var, x) **then return** failure

else return add $\{var/x\}$ to θ

Generalized Modus Ponens (GMP)

$$\frac{p_1', p_2', \dots, p_n', (p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q)}{q\theta} \quad \text{where } p_i'\theta = p_i \theta \text{ for all } i$$

p_1' is <i>King(John)</i>	p_1 is <i>King(x)</i>
p_2' is <i>Greedy(y)</i>	p_2 is <i>Greedy(x)</i>
θ is {x/John,y/John}	q is <i>Evil(x)</i>
$q \theta$ is <i>Evil(John)</i>	

- GMP used with KB of **definite clauses** (**exactly** one positive literal)
- All variables assumed universally quantified

Inference unification

$UNIFY(p, q) = \theta$ Where $SUBST(\theta, p) = SUBST(\theta, q)$

For example:

- ▶ We ask $ASKVARS(Knows(John, x))$ (Whom does John know?)
- ▶ $UNIFY(Knows(John, x), Knows(John, Jane)) = \{x/Jane\}$
- ▶ $UNIFY(Knows(John, x), Knows(y, Bill)) = \{y/John, x/Bill\}$
- ▶ $UNIFY(Knows(John, x), Knows(y, Mother(y))) = \{y/John, x/Mother(John)\}$

Algorithm in the book (goes variable by variable recursively unifying)

Soundness of GMP

- Need to show that

$$p_1', \dots, p_n', (p_1 \wedge \dots \wedge p_n \Rightarrow q) \models q\theta$$

provided that $p_i'\theta = p_i\theta$ for all i

- Lemma: For any sentence p , we have $p \models p\theta$ by UI

1. $(p_1 \wedge \dots \wedge p_n \Rightarrow q) \models (p_1 \wedge \dots \wedge p_n \Rightarrow q)\theta = (p_1\theta \wedge \dots \wedge p_n\theta \Rightarrow q\theta)$
2. $p_1', \dots, p_n' \models p_1' \wedge \dots \wedge p_n' \models p_1'\theta \wedge \dots \wedge p_n'\theta$
3. From 1 and 2, $q\theta$ follows by ordinary Modus Ponens

Example knowledge base

- The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American.
- Prove that Col. West is a criminal

Example knowledge base contd.

... it is a crime for an American to sell weapons to hostile nations:

R1: American(x) $\wedge$ Weapon(y) $\wedge$ Sells(x,y,z) $\wedge$ Hostile(z) $\Rightarrow$ Criminal(x)

Nono ... has some missiles, i.e., $\exists x$ Owns(Nono,x) $\wedge$ Missile(x):

R2: Owns(Nono,M₁) and Missile(M₁)

... all of its missiles were sold to it by Colonel West

R3: Missile(x) $\wedge$ Owns(Nono,x) $\Rightarrow$ Sells(West,x,Nono)

Missiles are weapons:

R4: Missile(x) $\Rightarrow$ Weapon(x)

An enemy of America counts as "hostile":

R5: Enemy(x,America) $\Rightarrow$ Hostile(x)

West, who is American ...

R6: American(West)

The country Nono, an enemy of America ...

R7: Enemy(Nono,America)

Example knowledge base contd.

R1: American(x) $\wedge$ Weapon(y) $\wedge$ Sells(x,y,z) $\wedge$ Hostile(z) $\Rightarrow$ Criminal(x)

R2: Owns(Nono,M₁) and Missile(M₁)

R3: Missile(x) $\wedge$ Owns(Nono,x) $\Rightarrow$ Sells(West,x,Nono)

R4: Missile(x) $\Rightarrow$ Weapon(x)

R5: Enemy(x,America) $\Rightarrow$ Hostile(x)

R6: American(West)

R7: Enemy(Nono,America)

Iteration 1

- R3 satisfied with {x/M1} and R8 will be generated and added to KB
 - R8: Sells(West,M1,Nono)
- R4 satisfied with {x/M1} and R9 will be generated and added to KB
 - R9: Weapon(M1)
- R5 satisfied with {x/Nono} and R10 is generated and added to KB
 - R10: Hostile(Nono)

Iteration 2

- Using R8 to R10, we can infer that R1 is satisfied with {x/west,y/M1,z/Nono} and Criminal(West) is added to KB.

Inference in first order Logic

- ▶ Once we have facts that evaluate to T or F
- ▶ We can apply Forward Chaining, Backwards Chaining and Resolution
- ▶ The key is to understand Unification
- ▶ Very similar to Logical agents.

Forward chaining algorithm

function FOL-FC-ASK(KB, α) **returns** a substitution or *false*

repeat until new is empty

$new \leftarrow \{ \}$

for each sentence r **in** KB **do**

$(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-APART}(r)$

for each θ **such that** $(p_1 \wedge \dots \wedge p_n)\theta = (p'_1 \wedge \dots \wedge p'_n)\theta$

for some $p'_1, \dots, p'_n$ **in** KB

$q' \leftarrow \text{SUBST}(\theta, q)$

if q' is not a renaming of a sentence already in KB or new **then do**

add q' **to** new

$\phi \leftarrow \text{UNIFY}(q', \alpha)$

if ϕ is not *fail* **then return** ϕ

add new **to** KB

return *false*

Forward chaining proof

R2: Owns(Nono, M₁) and Missile(M₁)

R6: American(West)

R7: Enemy(Nono, America)

American(West)

Missile(M1)

Owns(Nono, M1)

Enemy(Nono, America)

Forward chaining proof

R2: Owns(Nono,M₁) and Missile(M₁)

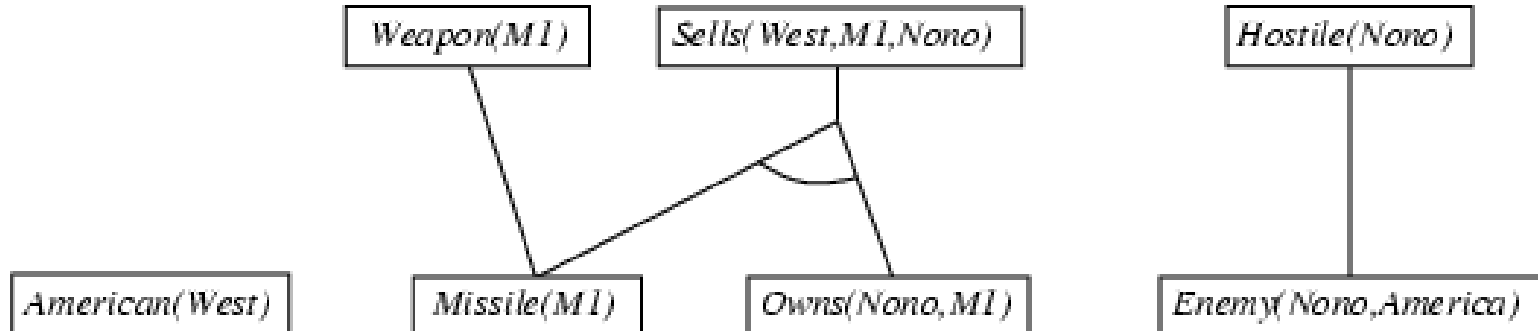
R6: American(West)

R7: Enemy(Nono,America)

R3: Missile(x) $\wedge$ Owns(Nono,x) $\Rightarrow$ Sells(West,x,Nono)

R4: Missile(x) $\Rightarrow$ Weapon(x)

R5: Enemy(x,America) $\Rightarrow$ Hostile(x)



Forward chaining proof

R2: $\text{Owns}(\text{Nono}, M_1)$ and $\text{Missile}(M_1)$

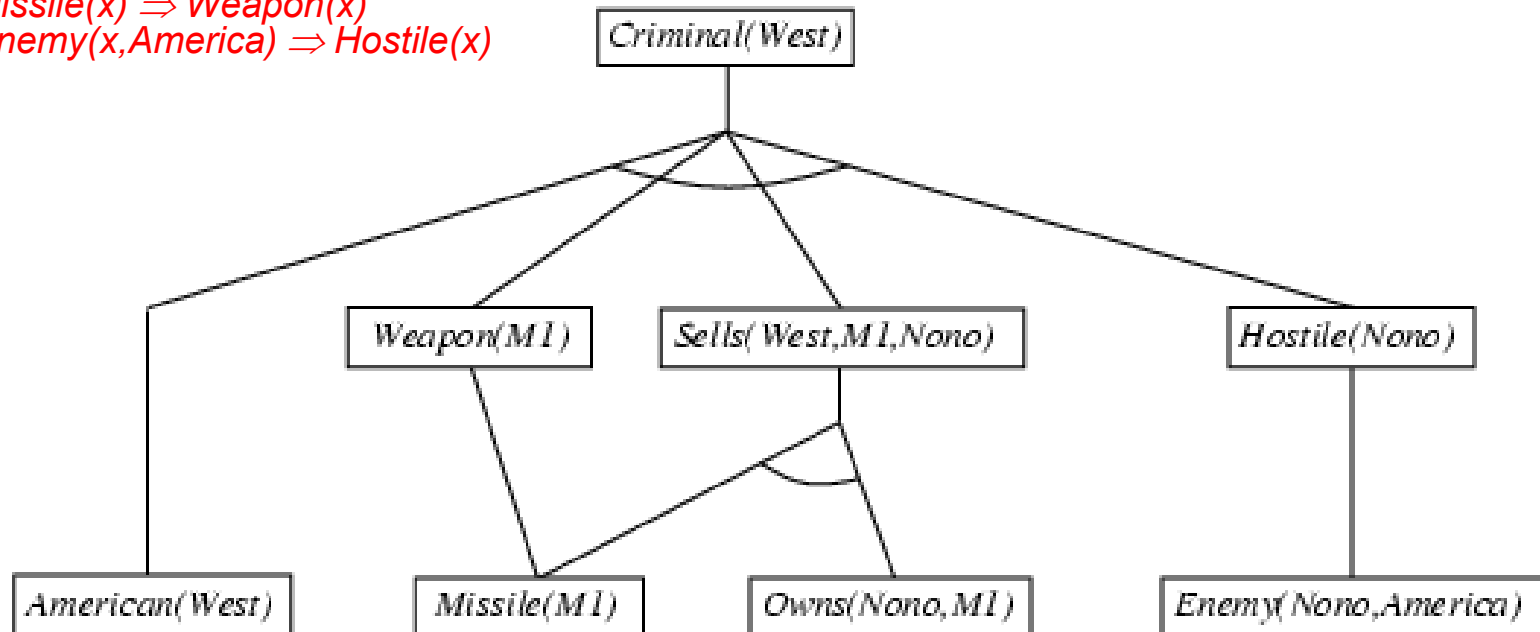
R6: $\text{American}(\text{West})$

R7: $\text{Enemy}(\text{Nono}, \text{America})$

R3: $\text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$

R4: $\text{Missile}(x) \Rightarrow \text{Weapon}(x)$

R5: $\text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$



R1: $\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$

Properties of forward chaining

- Sound and complete for first-order definite clauses
- **Datalog** = first-order definite clauses + **no functions**
- FC terminates for Datalog in finite number of iterations
- May not terminate in general if α is not entailed
- This is unavoidable: entailment with definite clauses is semidecidable

Efficiency of forward chaining

Incremental forward chaining: no need to match a rule on iteration k if a premise wasn't added on iteration $k-1$

⇒ match each rule whose premise contains a newly added positive literal

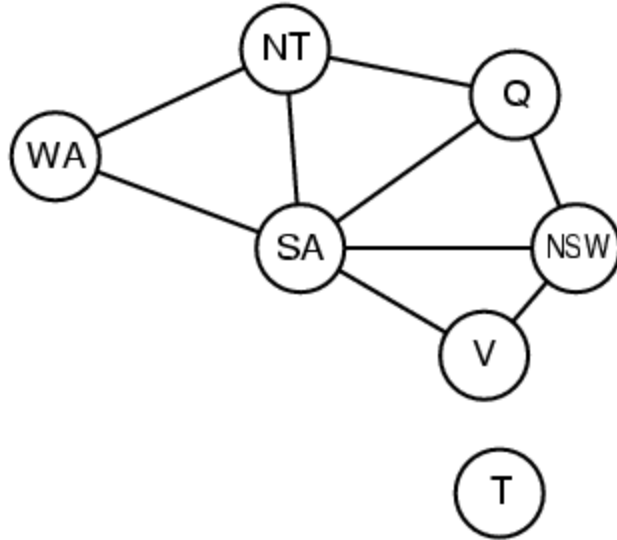
Matching itself can be expensive:

Database indexing allows $O(1)$ retrieval of known facts

– e.g., query $Missile(x)$ retrieves $Missile(M_1)$

Forward chaining is widely used in **deductive databases**

Hard matching example


$$\begin{aligned} & \text{Diff}(wa, nt) \wedge \text{Diff}(wa, sa) \wedge \text{Diff}(nt, q) \wedge \\ & \text{Diff}(nt, sa) \wedge \text{Diff}(q, nsw) \wedge \text{Diff}(q, sa) \wedge \\ & \text{Diff}(nsw, v) \wedge \text{Diff}(nsw, sa) \wedge \text{Diff}(v, sa) \Rightarrow \\ & \text{Colorable()} \end{aligned}$$
$$\begin{array}{ll} \text{Diff}(\text{Red}, \text{Blue}) & \text{Diff}(\text{Red}, \text{Green}) \\ \text{Diff}(\text{Green}, \text{Red}) & \text{Diff}(\text{Green}, \text{Blue}) \\ \text{Diff}(\text{Blue}, \text{Red}) & \text{Diff}(\text{Blue}, \text{Green}) \end{array}$$

- *Colorable()* is inferred iff the CSP has a solution
- CSPs include 3SAT as a special case, hence matching is NP-hard

Backward chaining algorithm

function FOL-BC-ASK(KB , goals, θ) returns a set of substitutions

inputs: KB , a knowledge base

$goals$, a list of conjuncts forming a query

θ , the current substitution, initially the empty substitution $\{ \}$

local variables: ans , a set of substitutions, initially empty

if $goals$ is empty **then return** $\{ \theta \}$

$q' \leftarrow \text{SUBST}(\theta, \text{FIRST}(goals))$

for each r in KB where $\text{STANDARDIZE-APART}(r) = (p_1 \wedge \dots \wedge p_n \Rightarrow q)$
and $\theta' \leftarrow \text{UNIFY}(q, q')$ succeeds

$ans \leftarrow \text{FOL-BC-ASK}(KB, [p_1, \dots, p_n | \text{REST}(goals)], \text{COMPOSE}(\theta, \theta')) \cup ans$

return ans

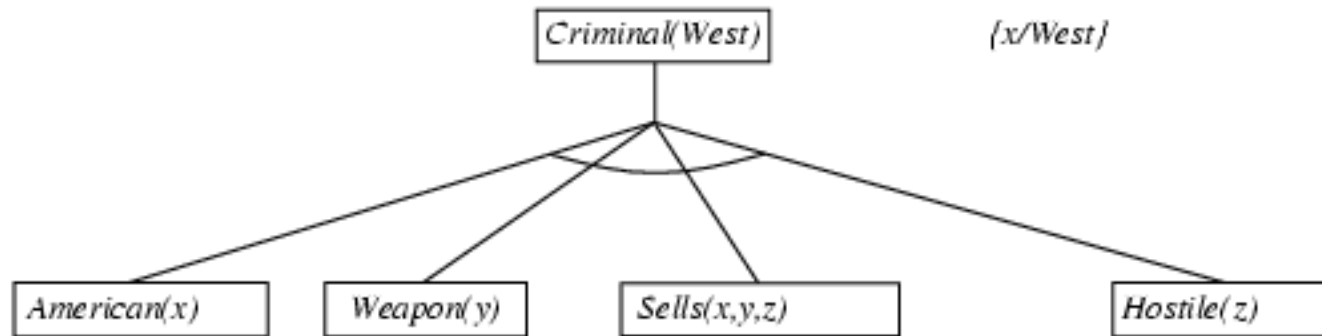
$$\text{SUBST}(\text{COMPOSE}(\theta_1, \theta_2), p) = \text{SUBST}(\theta_2, \text{SUBST}(\theta_1, p))$$

Backward chaining example

Criminal(West)

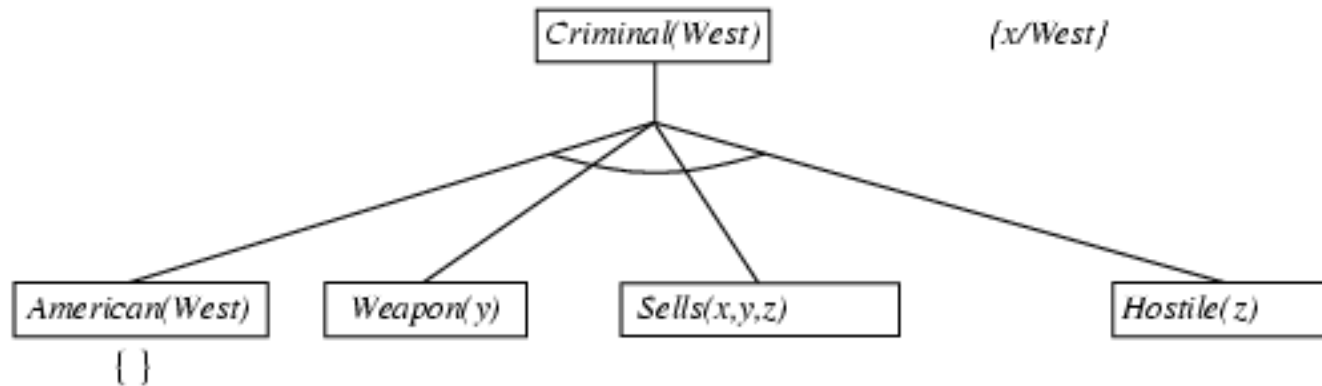
R1: $\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x,y,z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$

Backward chaining example



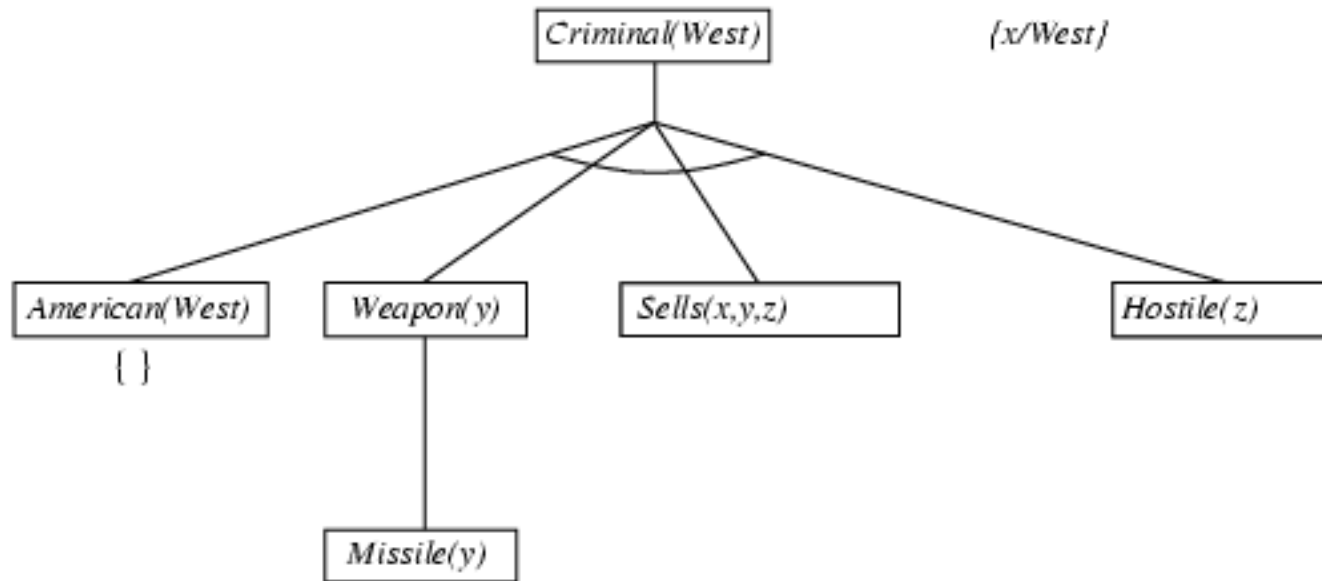
R1: $American(x) \wedge Weapon(y) \wedge Sells(x,y,z) \wedge Hostile(z) \Rightarrow Criminal(x)$

Backward chaining example



R1: $\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x,y,z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$

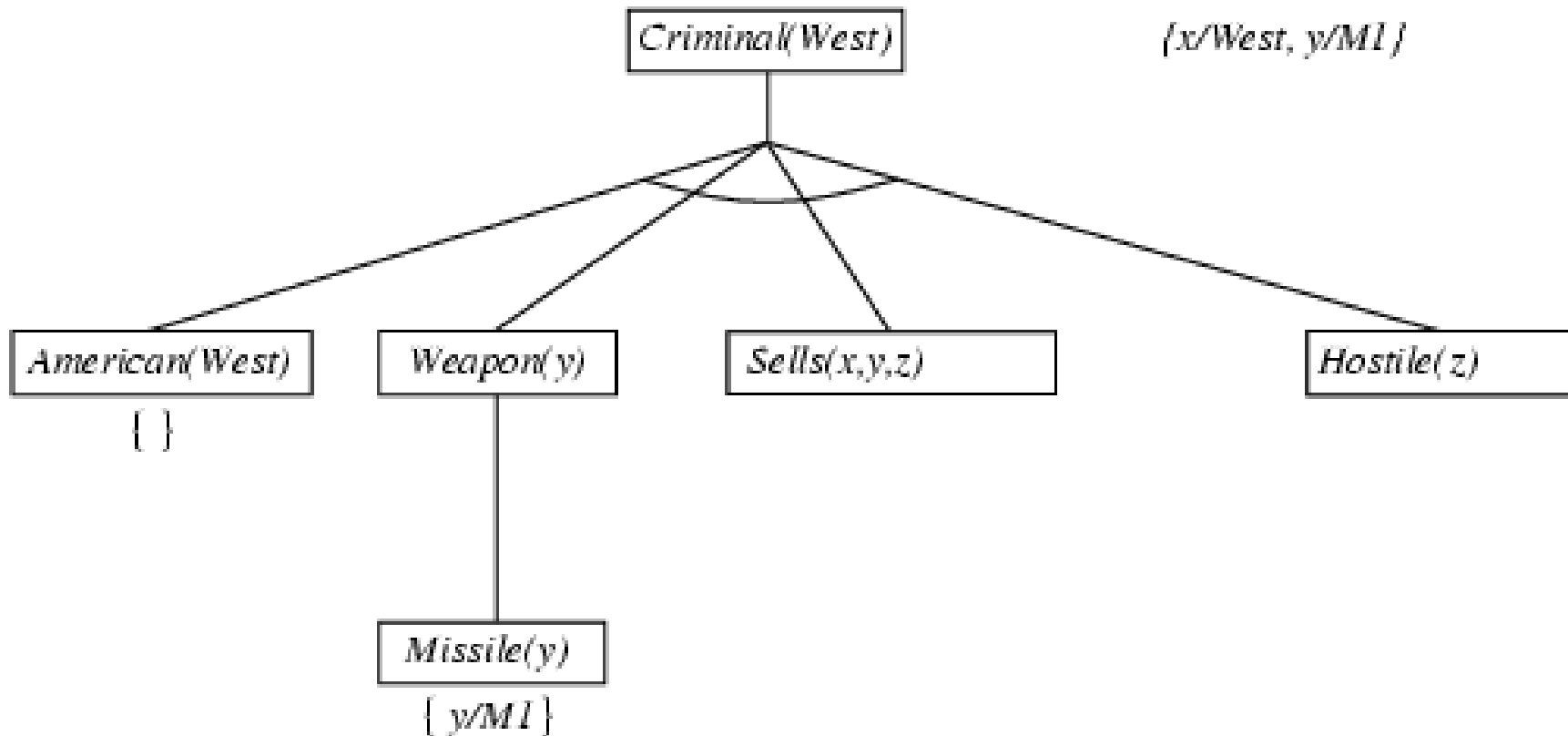
Backward chaining example



R4: $\text{Missile}(x) \Rightarrow \text{Weapon}(x)$

R1: $\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x,y,z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$

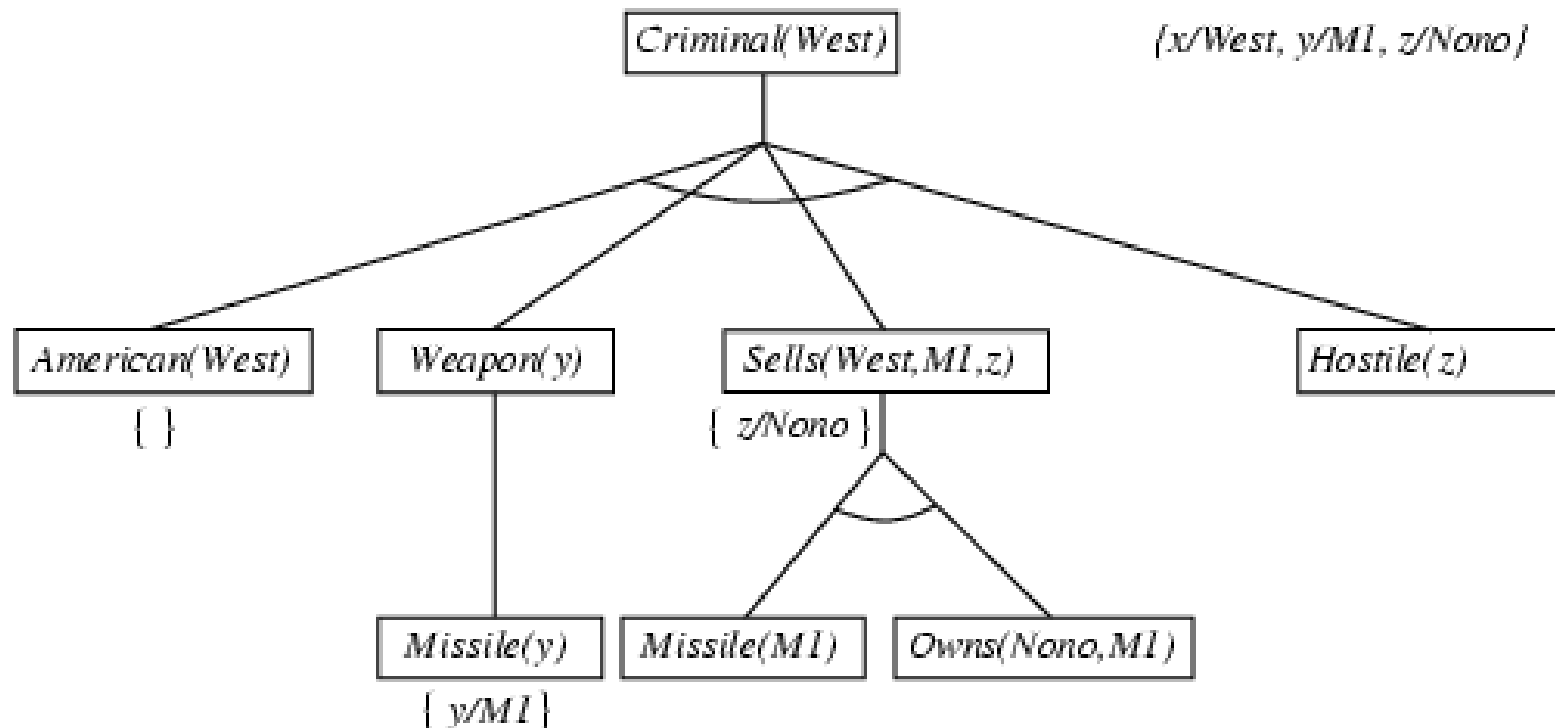
Backward chaining example



R4: $\text{Missile}(x) \Rightarrow \text{Weapon}(x)$

R1: $\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x,y,z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$

Backward chaining example

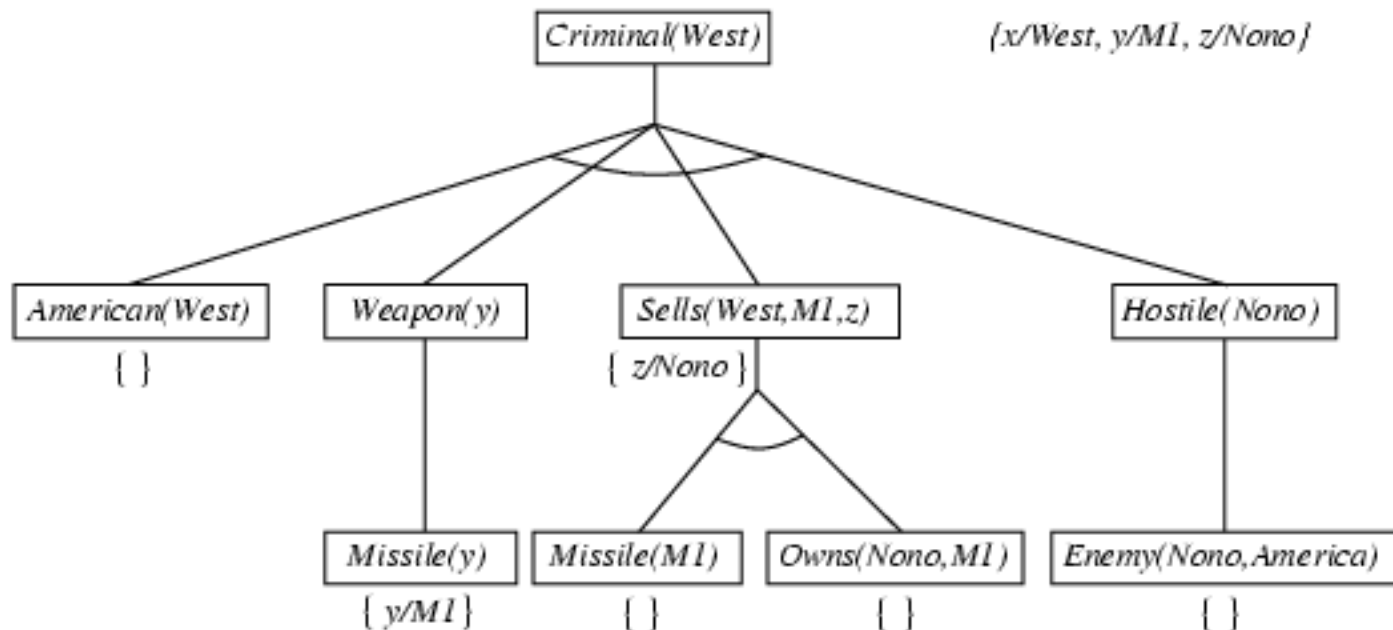


R3: $Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(West, x, Nono)$

R4: $Missile(x) \Rightarrow Weapon(x)$

R1: $American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$

Backward chaining example



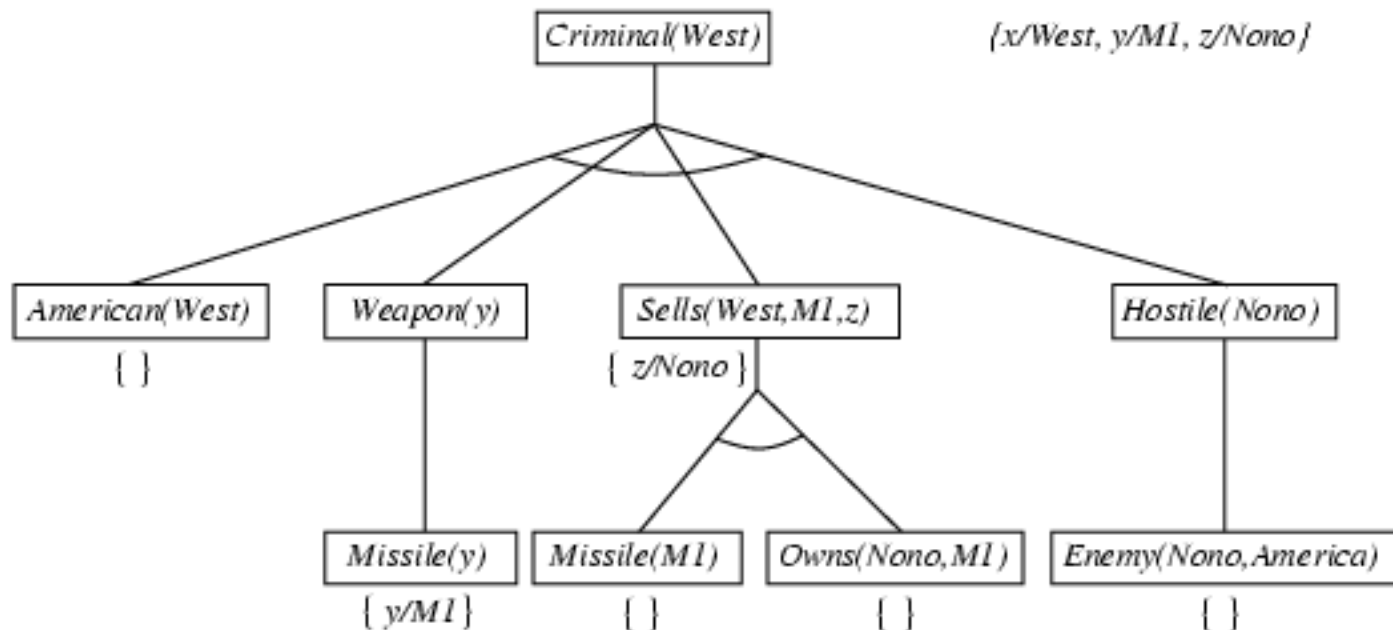
R5: $\text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$

R3: $\text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$

R4: $\text{Missile}(x) \Rightarrow \text{Weapon}(x)$

R1: $\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$

Backward chaining example



R5: $\text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$

R3: $\text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$

R4: $\text{Missile}(x) \Rightarrow \text{Weapon}(x)$

R1: $\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$

Properties of backward chaining

- **Depth-first recursive proof search:** space is linear in size of proof
- Incomplete due to infinite loops
⇒ fix by checking current goal against every goal on stack
- Inefficient due to repeated subgoals (both success and failure)
⇒ fix using caching of previous results (extra space)
- Widely used for **logic programming**

Logic programming: Prolog

- Algorithm = Logic + Control
- Basis: backward chaining with Horn clauses + bells & whistles
Widely used in Europe, Japan (basis of 5th Generation project)
Compilation techniques $\Rightarrow$ 60 million LIPS
- Program = set of clauses = `head :- literal1, ... literaln.`

`criminal(X) :- american(X), weapon(Y), sells(X,Y,Z), hostile(Z).`
- Depth-first, left-to-right backward chaining
- Built-in predicates for arithmetic etc., e.g., `X is Y*Z+3`
- Built-in predicates that have side effects (e.g., input and output predicates, assert/retract predicates)
- Closed-world assumption ("negation as failure")
 - e.g., given `alive(X) :- not dead(X).`
 - `alive(joe)` succeeds if `dead(joe)` fails

Prolog

- Appending two lists to produce a third:

```
append([ ], Y, Y) .
```

```
append([X|L], Y, [X|Z]) :- append(L, Y, Z) .
```

- query: `append(A, B, [1, 2]) ?`

- answers: `A = [ ]        B = [1, 2]`

`A = [1]        B = [2]`

`A = [1, 2]    B = [ ]`

Resolution: brief summary

- Full first-order version:

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \quad m_1 \vee \cdots \vee m_n}{(\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n)\theta}$$

where $\text{Unify}(\ell_i, \neg m_j) = \theta$.

- The two clauses are assumed to be standardized apart so that they share no variables.
- For example,

$$\frac{\neg \text{Rich}(x) \vee \text{Unhappy}(x) \quad \text{Rich}(\text{Ken})}{\text{Unhappy}(\text{Ken})}$$

with $\theta = \{x/\text{Ken}\}$

- Apply resolution steps to $\text{CNF}(\text{KB} \wedge \neg \alpha)$; complete for FOL

Conversion to CNF

- Everyone who loves all animals is loved by someone:

$$\forall x [\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x,y)] \Rightarrow [\exists y \text{ Loves}(y,x)]$$

- 1. Eliminate biconditionals and implications

$$\forall x [\neg \forall y \neg \text{Animal}(y) \vee \text{Loves}(x,y)] \vee [\exists y \text{ Loves}(y,x)]$$

- 2. Move $\neg$ inwards: $\neg \forall x p \equiv \exists x \neg p$, $\neg \exists x p \equiv \forall x \neg p$

$$\forall x [\exists y \neg(\neg \text{Animal}(y) \vee \text{Loves}(x,y))] \vee [\exists y \text{ Loves}(y,x)]$$

$$\forall x [\exists y \neg \neg \text{Animal}(y) \wedge \neg \text{Loves}(x,y)] \vee [\exists y \text{ Loves}(y,x)]$$

$$\forall x [\exists y \text{ Animal}(y) \wedge \neg \text{Loves}(x,y)] \vee [\exists y \text{ Loves}(y,x)]$$

Conversion to CNF contd.

3. Standardize variables: each quantifier should use a different one

$$\forall x [\exists y \textit{Animal}(y) \wedge \neg \textit{Loves}(x,y)] \vee [\exists z \textit{Loves}(z,x)]$$

4. Skolemize: a more general form of existential instantiation.
Each existential variable is replaced by a **Skolem function** of the enclosing universally quantified variables:

$$\forall x [\textit{Animal}(F(x)) \wedge \neg \textit{Loves}(x,F(x))] \vee \textit{Loves}(G(x),x)$$

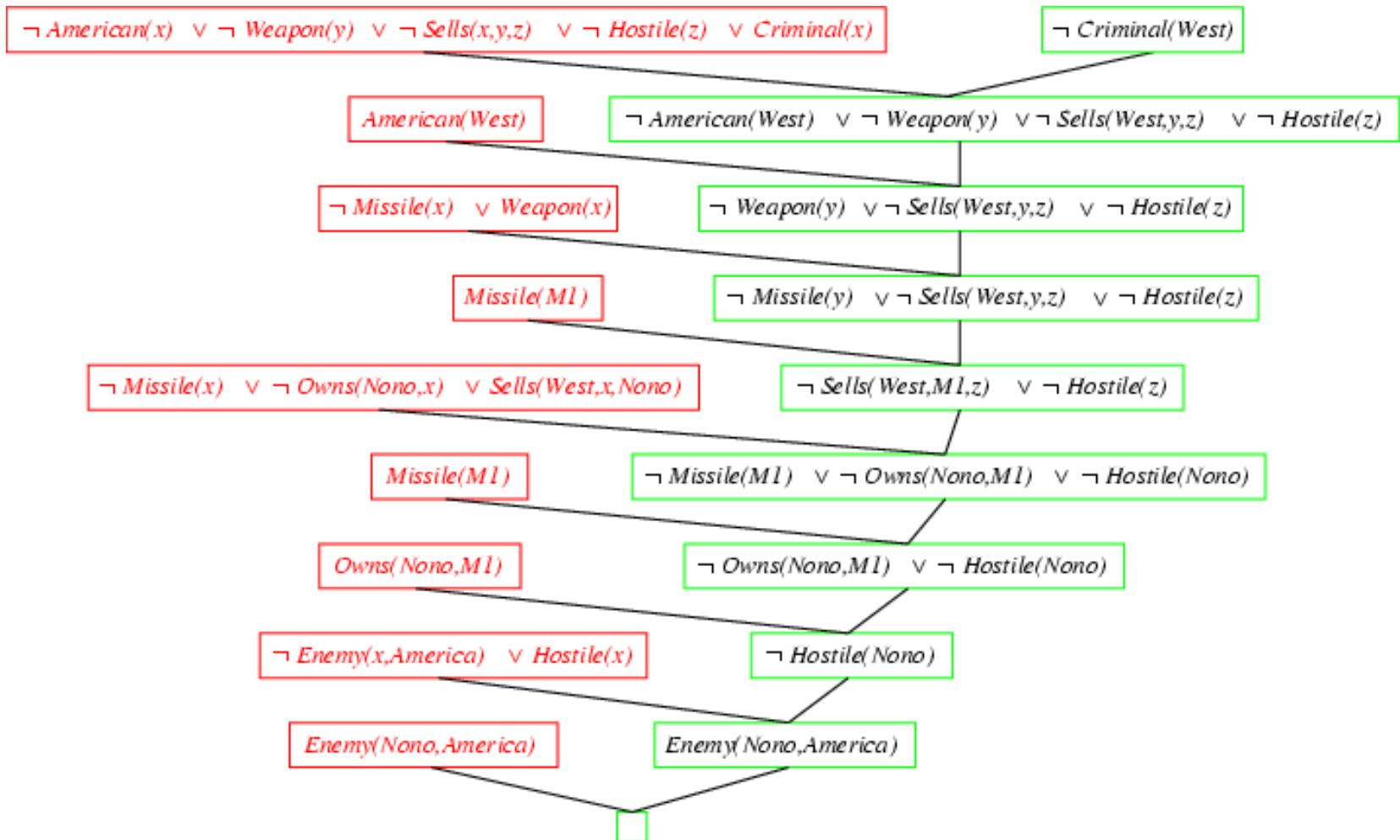
5. Drop universal quantifiers:

$$[\textit{Animal}(F(x)) \wedge \neg \textit{Loves}(x,F(x))] \vee \textit{Loves}(G(x),x)$$

6. Distribute $\vee$ over $\wedge$:

$$[\textit{Animal}(F(x)) \vee \textit{Loves}(G(x),x)] \wedge [\neg \textit{Loves}(x,F(x)) \vee \textit{Loves}(G(x),x)]$$

Resolution proof: definite clauses





University of Colorado
Colorado Springs



University of Colorado

Boulder | Colorado Springs | Denver | Anschutz Medical Campus